



Cálculo Numérico

Universidade Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

Capítulo 2

Resolução de sistemas de equações lineares

Resumo

O problema fundamental da Álgebra Linear é resolver sistemas lineares da forma $Ax = b$, assumindo que uma solução exista. Diversas abordagens podem ser empregadas para resolver esse problema, cada uma com vantagens e desvantagens específicas. Neste curso, utilizaremos o método de Gauss-Jordan para resolver sistemas lineares quadrados [1]. Também exploraremos a biblioteca `SciPy`, do `Python`, com foco em suas funcionalidades para álgebra linear e manipulação de matrizes esparsas [2]. No entanto, em nossa primeira implementação, utilizaremos apenas os recursos básicos da linguagem `Python` para demonstrar que os métodos matemáticos de Álgebra Linear podem ser adequadamente expressos como algoritmos computacionais.

“A matemática nos leva para o mundo das formas eternas.”

Platão

Introdução

Nesta unidade, estudamos a resolução de sistemas de n equações lineares com n incógnitas. Este é, de longe, o tema mais extenso e, sem dúvida, um dos mais importantes

do curso, pois diversos conceitos matemáticos e computacionais apresentados aqui serão reutilizados ao longo de outras unidades. Além disso, é praticamente impossível realizar análises numéricas sem encontrar sistemas de equações simultâneas [1].

Em muitos casos, é possível reduzir o tempo de execução e o uso de memória explorando propriedades específicas da matriz de coeficientes, como a **esparsidade** — situação em que a maioria dos elementos da matriz é nula. Por esse motivo, existem diversos algoritmos otimizados para resolver grandes conjuntos de equações, cada um adequado a diferentes tipos de matrizes: simétricas, esparsas, em faixas, entre outras [3].

Não abordaremos todos esses métodos neste momento, mas apresentaremos os fundamentos da eliminação de Gauss-Jordan e alguns algoritmos úteis aplicáveis a matrizes esparsas e estruturadas.

Forma Geral de um Sistema Linear

Um sistema linear com n equações e n incógnitas pode ser representado da seguinte forma:

$$\begin{cases} A_{11}x_1 + A_{12}x_2 + \dots + A_{1n}x_n & b_1 \\ A_{21}x_1 + A_{22}x_2 + \dots + A_{2n}x_n & b_2 \\ \vdots & \vdots \\ A_{n1}x_1 + A_{n2}x_2 + \dots + A_{nn}x_n & b_n \end{cases} =$$

Na notação matricial, o sistema é escrito como:

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b}, \quad \text{ou seja:} \quad \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1n} \\ A_{21} & A_{22} & \dots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \dots & A_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

onde os coeficientes A_{ij} e as constantes b_j são conhecidos, e x_i representa as incógnitas.

Um sistema de n equações lineares em n incógnitas possui solução única se, e somente se, o determinante da matriz de coeficientes for diferente de zero, ou seja, a matriz for não singular ($|\mathbf{A}| \neq 0$). As linhas e colunas de uma matriz não singular são line-

armente independentes no sentido de que nenhuma linha (ou coluna) é uma combinação linear de outras linhas (ou colunas).

Se a matriz coeficiente for singular, as equações podem ter um número infinito de soluções, ou nenhuma solução, dependendo do vetor constante. Como ilustração, tomemos as seguintes equações:

$$2x + y = 3 \quad 4x + 2y = 6$$

Como a segunda equação pode ser obtida multiplicando a primeira equação por dois, qualquer combinação de x e y que satisfaça a primeira equação também é uma solução da segunda equação. O número de tais combinações é infinito. Por outro lado, as equações:

$$2x + y = 3 \quad 4x + 2y = 0$$

não tem solução porque a segunda equação, sendo equivalente a $2x + y = 0$, contradiz a primeira. Portanto, qualquer solução que satisfaça uma equação não pode satisfazer a outra.

Agora, tomemos um outro exemplo:

$$\begin{cases} x + y + 2z = 9 \\ 2x + 4y - 3z = 1 \\ 3x + 6y - 5z = 0 \end{cases} \quad (2.1)$$

A solução do sistema linear acima pode ser obtida de maneira muito simples. A fim de eliminarmos a variável x nessa equação, podemos multiplicar a primeira por -2 e adicionarmos à segunda. Depois, multiplicamos novamente a primeira por -3 e adicionamos à terceira equação. Essas operações constituem o que chamamos de combinação linear e são pertinentes a solução do problema. Implementando essas operações iremos obter:

$$\begin{cases} x + y + 2z = 9 \\ 2y - 7z = -17 \\ 3y - 11z = -27 \end{cases}$$

Observe que a segunda e a terceira equações formam um sistema de duas equações nas incógnitas y e z . Novamente, podemos eliminar a variável y multiplicando a segunda equação por 3 e a terceira por 2 e efetuando a subtração da segunda equação pela terceira,

obtendo:

$$\begin{cases} x + y + 2z = 9 \\ 6y - 21z = -51 \\ z = 3 \end{cases}$$

encontrando assim o valor de z , a saber $z = 3$. Substituindo nas duas primeiras equações, temos: $6y = 63 - 51 = 12$, ou $y = 2$, e finalmente, $x = 9 - 2 - 6 = 1$.

Portanto, temos que a solução do sistema dado é $x = 1$, $y = 2$ e $z = 3$, que neste caso é a única solução! $\triangleleft$

Como outro exemplo, vamos resolver o seguinte sistema:

$$\begin{cases} x + y + z = 1 \\ x + y - z = 0 \\ 2x + 2y = 1 \end{cases} \quad (2.2)$$

Procedendo como anteriormente, multiplicamos a segunda equação por -2 e adicionamos à terceira para eliminarmos x na terceira equação. Em seguida, subtraímos a segunda equação da primeira para eliminarmos x na segunda equação, obtendo:

$$\begin{cases} x + y + z = 1 \\ 2z = 1 \\ 2z = 1 \end{cases}$$

Neste caso, ao eliminar a variável x , eliminamos também a variável y e obtemos a segunda equação idêntica à terceira. O sistema é, assim, equivalente ao seguinte:

$$\begin{cases} x + y + z = 1 \\ 2z = 1 \end{cases}$$

Facilmente, encontramos que $z = 1/2$. Porém, agora temos algo distinto do primeiro exemplo. Observe que para $z = 1/2$, $x = -t + 1/2$ é sempre solução para qualquer valor de $y = t$. Vemos, portanto, que o sistema possui infinitas soluções, ou seja: $x = -t + 1/2$, $y = t$ e $z = 1/2$. $\triangleleft$

Vejam que da forma que obtivemos as soluções nos exemplos anteriores, não temos exatamente um modelo de algoritmo para executar a busca das soluções de sistemas lineares.

Por isso, precisamos aplicar em tais problemas o método da eliminação gaussiana se quisermos uma primeira modelagem computacional, que basicamente funciona a partir da aplicação três operações lineares: **1. multiplicar uma linha por um valor escalar**, **2. trocar duas linhas** ou **3. adicionar um múltiplo escalar de uma linha a outra**. Vamos retornar ao sistema linear apresentada na Equação (2.1).

$$\begin{cases} x + y + 2z = 9 \\ 2x + 4y - 3z = 1 \\ 3x + 6y - 5z = 0 \end{cases}$$

A abordagem para resolver o sistema linear acima pelo método de Gauss exige escrever uma matriz associada ao sistema da seguinte forma: **cada linha da matriz corresponderá a uma das equações**. Teremos, portanto, uma matriz com 3 linhas. Cada coeficiente da primeira equação corresponderá ordenadamente a uma entrada da primeira linha. O termo independente será a 4ª entrada desta primeira linha. Ela terá 4 entradas. Para a segunda e terceira linhas procedemos da mesma forma. Teremos, portanto, uma matriz 3×4 associada ao sistema, chamada de **matriz aumentada**, ou seja:

$$[Ab] = \left[\begin{array}{ccc|c} 1 & 1 & 2 & 9 \\ 2 & 4 & -3 & 1 \\ 3 & 6 & -5 & 0 \end{array} \right] \tag{2.3}$$

A matriz acima se chama aumenta para se distinguir da matriz:

$$A = \left[\begin{array}{ccc} 1 & 1 & 2 \\ 2 & 4 & -3 \\ 3 & 6 & -5 \end{array} \right]$$

denominada **matriz de coeficientes**. A metodologia padrão para resolver o sistema linear $Ax = b$ analiticamente é reduzir a matriz aumentada $[Ab]$ para a forma escalonada por linha (*Row Echelon Form* - REF) via *eliminação gaussiana*, e ao final utilizamos uma substituição reversa a fim de encontra o valor de cada uma das variáveis do sistema.

NOTA

Uma matriz $m \times n$ se diz na forma escalonada reduzida por linhas quando:

1. Se uma linha não consistir só de zeros, então o primeiro número não

nulo da linha é 1. Chamamos este número 1 de **líder** ou **pivô**.

2. Se existirem linhas constituídas somente de zeros, elas estão agrupadas nas linhas inferiores da matriz.
3. Em quaisquer duas linhas sucessivas que não consistem só de zeros, o líder da linha inferior ocorre mais à direita que o líder da linha superior.
4. Cada coluna que contém um líder tem zeros nas demais entradas.

Se apenas a condição 4 não é satisfeita, a matriz se diz na forma escalonada.

A matriz aumentada já satisfaz a **condição 1** (o pivô da primeira linha está na posição (1, 1) com valor 1). Aplicamos agora operações elementares para atender à **condição 2**: zerar os elementos abaixo desse pivô.

$$[Ab] = \left[\begin{array}{ccc|c} 1 & 1 & 2 & 9 \\ 2 & 4 & -3 & 1 \\ 3 & 6 & -5 & 0 \end{array} \right] \begin{array}{l} L_2 \leftarrow L_2 - 2L_1 \\ L_3 \leftarrow L_3 - 3L_1 \end{array}$$

encontramos:

$$[Ab] = \left[\begin{array}{ccc|c} 1 & 1 & 2 & 9 \\ 0 & 2 & -7 & -17 \\ 0 & 3 & -11 & -27 \end{array} \right] L_2 \leftarrow \frac{1}{2}L_2$$

Para obter o próximo pivô (na posição (2, 2)), normalizamos a segunda linha conforme já indicamos acima:

$$[Ab] = \left[\begin{array}{ccc|c} 1 & 1 & 2 & 9 \\ 0 & 1 & -\frac{7}{2} & -\frac{17}{2} \\ 0 & 3 & -11 & -27 \end{array} \right] L_3 \leftarrow L_3 - 3L_2$$

Zeramos agora o elemento abaixo do pivô com a transformação linear indicada:

$$[Ab] = \left[\begin{array}{ccc|c} 1 & 1 & 2 & 9 \\ 0 & 1 & -\frac{7}{2} & -\frac{17}{2} \\ 0 & 0 & -\frac{1}{2} & -\frac{3}{2} \end{array} \right] L_3 \leftarrow -2L_3$$

Por fim, para obter o pivô da terceira linha igual a 1, multiplicamos a terceira linha da matriz por (-2) , conforme indicação:

$$[Ab] = \left[\begin{array}{ccc|c} 1 & 1 & 2 & 9 \\ 0 & 1 & -\frac{7}{2} & -\frac{17}{2} \\ 0 & 0 & 1 & 3 \end{array} \right]$$

Neste ponto, atingimos a **forma escalonada por linhas (REF)**, e encerramos o **método de Gauss**. Por substituição regressiva, encontramos:

$$x = 1, \quad y = 2, \quad z = 3 \quad \triangleleft$$

Para obtermos a **forma escalonada reduzida por linhas (Row Reduced Echelon Form - RREF)** – exatamente a contribuição de Jordan ao método, continuamos aplicando operações para que os elementos acima de cada pivô também sejam zerados – e isso caracteriza o complemento do **método de Gauss-Jordan**.

Começamos a partir do pivô da última linha que contém duas entradas acima dele que precisam ser zeradas. As transformações lineares exigidas são indicadas abaixo:

$$[Ab] = \left[\begin{array}{ccc|c} 1 & 1 & 2 & 9 \\ 0 & 1 & -7/2 & -17/2 \\ 0 & 0 & 1 & 3 \end{array} \right] \quad \begin{array}{l} L_1 \leftarrow L_1 - 2L_3 \\ L_2 \leftarrow L_2 + \frac{7}{2}L_3 \end{array}$$

cujo resultado é:

$$[Ab] = \left[\begin{array}{ccc|c} 1 & 1 & 0 & 3 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array} \right] \quad L_1 \leftarrow L_1 - L_2$$

Deixamos indicado acima a transformação linear necessária para zerar o elemento acima do pivô da segunda linha, obtendo após sua aplicação a matriz aumentada agora está na forma desejada:

$$L_1 \leftarrow L_1 - L_2 \Rightarrow [Ab] = \left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array} \right]$$

À esquerda da barra temos a chamada **matriz identidade** I , e a parte à direita contém a solução única do sistema:

$$x = 1, \quad y = 2, \quad z = 3$$

O método de Gauss-Jordan, além de fornecer uma forma sistemática para resolver sistemas lineares, é especialmente eficiente do ponto de vista algorítmico. Ao final de sua aplicação, obtemos diretamente a **forma escalonada reduzida por linhas** (RREF), com a matriz identidade à esquerda da barra e a solução explícita do sistema linear à direita.

Diferentemente do método de Gauss simples, que exige uma etapa posterior de substituição regressiva, o método completo de Gauss-Jordan **já incorpora todas as operações algébricas necessárias** durante o processo. Assim, ao final da execução do algoritmo, **nenhum passo adicional é necessário** para extrair a solução. Essa característica o torna particularmente adequado para implementação computacional, especialmente em situações em que se deseja automatizar completamente a resolução de sistemas — como é comum em softwares de álgebra linear, planilhas eletrônicas ou aplicações numéricas iterativas [4].

Análise de Sistemas Lineares: Bem Posto, Mal Posto e Mal Condicionado

Consideramos um sistema linear **bem posto** — na perspectiva matemática e dentro do contexto da álgebra linear (como referenciado por Strang, 2009 [4]) — quando ele satisfaz as condições do **Teorema de Hadamard** para sistemas lineares $Ax = b$, ou seja:

- **Existência:** O sistema tem pelo menos uma solução.
- **Unicidade:** O sistema tem no máximo uma solução (solução única).
- **Estabilidade:** Pequenas perturbações nos dados de entrada (como b ou A) resultam em pequenas mudanças na solução x .

No caso de um sistema linear, a **existência** e a **unicidade** estão atrelados ao fato da matriz A ser **não singular**, ou seja, seu determinante deve ser diferente de zero ($\det(A) \neq 0$).

Quanto a **estabilidade**, veremos a frente que essa característica (relacionada à perspectiva numérica), funcionará como um parâmetro secundário de classificação de sistemas lineares, cujas soluções estarão relacionadas à estabilidade numérica diante de pequenas perturbações nos dados de entrada (como erros de arredondamento em cálculos computacionais) que causam grandes variações nos resultados finais — demonstrando que o sistema pode estar **mal condicionado**. Isso será investigado através do estudo do **número de condição** da matriz.

Por outro lado, um sistema **mal posto** é aquele que falha em pelo menos uma das condições de Hadamard. Isso pode ocorrer se:

- A matriz $\mathbf{A}$ for singular ($\det(\mathbf{A}) = 0$), levando a nenhuma solução ou infinitas soluções.
- Mesmo que $\det(\mathbf{A}) \neq 0$, o sistema pode ser numericamente mal condicionado, ou seja, pequenas perturbações nos dados de entrada (como erros de arredondamento em cálculos computacionais) causam grandes variações na solução. Isso está relacionado ao número de condição da matriz.

Para compreender como essas propriedades — existência, unicidade e estabilidade — se manifestam na prática, é essencial analisar o comportamento da matriz $\mathbf{A}$ em termos de seu determinante e número de condição. Enquanto a não singularidade ($\det(\mathbf{A}) \neq 0$) garante a existência e unicidade da solução, o número de condição revela o grau de sensibilidade do sistema a perturbações numéricas. Essa análise torna-se particularmente crítica em casos onde a matriz $\mathbf{A}$ é singular ou quase singular, situações que amplificam as instabilidades numéricas e desafiam a resolução confiável do sistema, como será explorado a seguir.

Singularidade e Quase Singularidade

Uma questão fundamental na análise de sistemas lineares é entender o que ocorre quando a matriz de coeficientes $\mathbf{A}$ é quase singular – ou seja, quando o determinante $|\mathbf{A}|$ é aproximadamente zero. Nessas condições, pode ser complicado encontrar solução para o sistema devido a presença de instabilidades numéricas, em que pequenas alterações nos

coeficientes geram grandes variações nas soluções [5, 4].

Conforme já introduzimos, um sistema de n equações lineares em n incógnitas possui solução única se, e somente se, o determinante da matriz de coeficientes for diferente de zero, ou seja, se a matriz for **não singular**:

$$|\mathbf{A}| \neq 0 \quad (2.4)$$

Esse critério, embora simples, tem implicações profundas: se a matriz é singular, não há como aplicar algoritmos determinísticos – como eliminação de Gauss ou Gauss-Jordan – esperando obter uma solução única para o sistema. Nessas situações, ou o sistema é incompatível (sem solução), ou possui infinitas soluções.

Portanto, a verificação da condição de não singularidade da matriz de coeficientes não é apenas uma etapa preliminar: ela representa um divisor de águas entre problemas bem definidos e situações onde a resolução exige abordagens complementares, como análise de consistência, uso de métodos de mínimos quadrados ou regularização numérica.

Em contextos computacionais, mesmo quando o determinante não é exatamente zero, mas assume valores muito pequenos, o sistema pode se tornar numericamente instável: pequenas flutuações nos dados podem produzir soluções drasticamente diferentes, comprometendo a confiabilidade dos resultados.

Para avaliar se uma matriz está próxima da singularidade, uma abordagem comum consiste em comparar o valor absoluto de seu determinante com alguma medida que **represente** o “tamanho” da matriz. Essa medida é fornecida por uma **norma matricial**, que pode ser definida de diferentes maneiras. Uma das mais utilizadas é a **norma de Frobenius**, definida por:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n |a_{ij}|^2} \quad (2.5)$$

Essa norma fornece uma noção da magnitude global dos elementos da matriz e permite avaliar a proporção entre o determinante e a estrutura geral da matriz.

A seguir, analisaremos dois exemplos para ilustrar a aplicação da norma, bem como as conclusões que podem ser extraídas a partir dessa análise.

Exemplo 1 – Sistema bem posto

Sendo assim, considere a matriz de coeficientes do sistema linear já discutido:

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 2 \\ 2 & 4 & -3 \\ 3 & 6 & -5 \end{bmatrix}$$

Calculando a **norma de Frobenius** da matriz, obtemos:

$$\|\mathbf{A}\|_F = \sqrt{1^2 + 1^2 + 2^2 + 2^2 + 4^2 + (-3)^2 + 3^2 + 6^2 + (-5)^2} = \sqrt{105} \approx 10,25$$

Em seguida, calculamos o **determinante** da matriz $\mathbf{A}$ utilizando a **regra de Sarrus**, uma técnica específica para matrizes 3×3 , encontramos:

$$|\mathbf{A}| = 1 \cdot (4 \cdot (-5) - (-3) \cdot 6) - 1 \cdot (2 \cdot (-5) - (-3) \cdot 3) + 2 \cdot (2 \cdot 6 - 4 \cdot 3) = -1$$

Assim, obtemos $|\mathbf{A}| = -1 \neq 0$, o que indica que a matriz é **não singular** e, portanto, o sistema possui **solução única** [4]. $\triangleleft$

Apesar da simplicidade da regra de Sarrus, nesta disciplina de *Cálculo Numérico* adotaremos exclusivamente o **método de Gauss** para o cálculo do determinante, pois ele é mais adequado à implementação computacional e generalizada para matrizes de qualquer ordem.

Sendo assim, vamos refazer o cálculo do determinante da matriz proposta, aplicando o método da eliminação de Gauss, **sem normalização dos pivôs**. Considerando as primeiras transformações lineares necessárias, temos então:

$$|\mathbf{A}| = \begin{vmatrix} 1 & 1 & 2 \\ 2 & 4 & -3 \\ 3 & 6 & -5 \end{vmatrix} \xrightarrow{L_2 \leftarrow L_2 - 2L_1, L_3 \leftarrow L_3 - 3L_1} \begin{vmatrix} 1 & 1 & 2 \\ 0 & 2 & -7 \\ 0 & 3 & -11 \end{vmatrix}$$

Prosseguindo com a eliminação:

$$L_3 \leftarrow L_3 - \frac{3}{2}L_2 \Rightarrow \begin{vmatrix} 1 & 1 & 2 \\ 0 & 2 & -7 \\ 0 & 0 & -\frac{1}{2} \end{vmatrix}$$

Pelo método de Gauss, o determinante de uma matriz triangular é dado pelo produto dos elementos da diagonal principal, conforme a propriedade:

$$\det(A) = (-1)^s \prod_{i=1}^n a_{ii} \quad (2.6)$$

onde s representa o número de trocas de linha – caso tenha sido necessário. Portanto, aplicando a expressão acima, encontramos novamente o valor do determinante do sistema linear estudado:

$$|\mathbf{A}| = 1 \cdot 2 \cdot \left(-\frac{1}{2}\right) = -1$$

confirmando o resultado obtido anteriormente.

Como nossa intenção é implementar o método de **Gauss-Jordan** para a resolução de sistemas lineares, **focaremos apenas em soluções utilizando o método de Gauss no cálculo do determinante**, de modo a garantir uniformidade e alinhamento com os objetivos do curso. ◀

A norma de Frobenius pode servir como uma referência de escala para avaliar a magnitude relativa do determinante. No caso da matriz $\mathbf{A}$ aqui em estudo, embora o valor absoluto de seu determinante seja $|\det(\mathbf{A})| = 1$, ele é numericamente pequeno em comparação com a norma $\|\mathbf{A}\|_F = \sqrt{105} \approx 10,25$.

Uma medida que, em alguns contextos, é utilizada para dar uma ideia grosseira da relação entre essas grandezas é a razão:

$$\frac{|\det(\mathbf{A})|}{\|\mathbf{A}\|_F} = \frac{1}{\sqrt{105}} \approx 0,098$$

Apesar de essa razão estar abaixo de 1, ela ainda está relativamente distante de zero — concluindo que, quanto mais distante de zero, mais bem posto é o sistema. No próximo exemplo, de um sistema mal posto, veremos que essa razão é bem menor!

É importante destacar que a métrica (razão) acima **não é suficiente** para avaliar o condicionamento numérico do sistema — ou seja, sua estabilidade numérica. Isso porque um determinante pequeno pode ocorrer mesmo em sistemas bem condicionados, assim como sistemas mal condicionados podem ter determinantes moderados. No entanto, ela pode nos servir como mais uma ferramenta de diagnóstico inicial.

A análise mais adequada da sensibilidade do sistema envolve o **número condição** (κ), que considera tanto a norma da matriz quanto a da sua inversa. Como será discutido a frente, essa análise revelará que o sistema aqui utilizado como exemplo é, de fato, **mal condicionado** — sujeito, portanto, a instabilidades numéricas, — apesar de ser matematicamente **bem posto** — ou seja, possuir solução única garantida por $\det(\mathbf{A}) \neq 0$ [4].

Exemplo 2 – Sistema mal posto

Este segundo exemplo apresenta um sistema linear que exibe um comportamento característico de um **problema mal posto**: a solução varia drasticamente mesmo sob pequenas alterações nos coeficientes. Isso compromete a confiabilidade dos métodos numéricos e ilustra os limites da análise algébrica quando aplicada sem considerar os aspectos computacionais. Vejamos o sistema:

$$\begin{cases} 2x + y = 3 \\ 2x + 1,001y = 0 \end{cases}$$

A matriz de coeficientes é:

$$\mathbf{B} = \begin{bmatrix} 2 & 1 \\ 2 & 1,001 \end{bmatrix}$$

Calculando seu determinante:

$$|\mathbf{B}| = 2 \cdot 1,001 - 2 \cdot 1 = 0,002$$

E a norma de Frobenius:

$$\|\mathbf{B}\|_F = \sqrt{2^2 + 1^2 + 2^2 + (1,001)^2} = \sqrt{10,002001} \approx 3,16$$

Podemos agora calcular a razão entre o valor absoluto do determinante e a norma da matriz:

$$\frac{||\mathbf{B}||}{\|\mathbf{B}\|_F} = \frac{0,002}{3,16} \approx 0,00063$$

Esse valor é **bem mais próximo de zero**, sugerindo que a matriz está quase singular e, portanto, sujeita a instabilidades numéricas.

Podemos verificar facilmente, neste caso, as consequências dessa instabilidade numérica. Ao resolver o sistema, obtemos:

$$x = 1501,5; \quad y = -3000$$

Agora, alterando levemente o coeficiente $1,001 \rightarrow 1,002$, temos:

$$|B'| = 2 \cdot 1,002 - 2 \cdot 1 = 0,004$$

E a nova solução passa a ser:

$$x = 751,5; \quad y = -1500$$

Note que uma alteração de apenas 0,1% em um coeficiente resultou em uma variação de 50% na solução. Isso caracteriza um sistema com **condicionamento numérico extremamente desfavorável**, mesmo não sendo estritamente singular. Ou seja, pequenos erros de arredondamento em sistemas como este podem comprometer severamente a confiabilidade dos resultados numéricos. *Neste caso, conseguimos verificar diretamente as consequências dessa instabilidade numérica.*

Na sequência, vamos reavaliar estes dois exemplos sob a ótica do **mal condicionamento**, utilizando como ferramenta de análise o seu **número de condição** (κ).

Impacto no Cálculo Numérico

Sistemas mal condicionados são perigosos do ponto de vista computacional, pois erros de arredondamento inevitáveis em aritmética de ponto flutuante podem causar grandes distorções nos resultados. Como discutem Burden e Faires (2011) [6] e Chapra e Canale (2010) [7], o efeito acumulado desses erros pode comprometer seriamente a confiabilidade da solução.

Por isso, quando se suspeita de mau condicionamento, é prudente:

- Calcular o determinante de **A**;
- Estimar a norma de **A** e o seu **número de condição** (κ);
- Aplicar métodos numéricos mais estáveis (e.g., decomposição LU com pivotamento);

- Avaliar a sensibilidade por meio de perturbações nos coeficientes.

De fato, a forma mais direta de avaliar o comportamento de uma matriz em termos de estabilidade numérica é por meio do **número de condição**, definido como:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| \quad (2.7)$$

onde $\|\cdot\|$ representa uma norma matricial. Neste contexto, continuaremos a usar a **norma de Frobenius**.

O número de condição nos fornecerá uma medida da magnitude global dos elementos da matriz, nos permitindo estimar o quanto a solução pode variar em função de pequenas alterações nos dados.

Revisão do Exemplo 1 – Sistema bem posto, porém mal condicionado

Considere novamente a matriz de coeficientes do sistema discutido anteriormente:

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 2 \\ 2 & 4 & -3 \\ 3 & 6 & -5 \end{bmatrix}$$

Já havíamos calculado sua norma de Frobenius:

$$\|\mathbf{A}\|_F \approx 10,25$$

Para calcular o número de condição, precisamos também da norma de Frobenius da inversa de $\mathbf{A}$, denotada por $\|\mathbf{A}^{-1}\|_F$.

A inversa da matriz pode ser obtida pelo método de Gauss – Jordan, que consiste em montar uma matriz aumentada com $\mathbf{A}$ à esquerda e a matriz identidade $\mathbf{I}$ à direita. Aplicando operações elementares até que o lado esquerdo se torne a identidade, o lado direito resultará na matriz inversa procurada. Sendo assim nossa configuração inicial é a seguinte:

$$A^{-1} = \left[\begin{array}{ccc|ccc} 1 & 1 & 2 & 1 & 0 & 0 \\ 2 & 4 & -3 & 0 & 1 & 0 \\ 3 & 6 & -5 & 0 & 0 & 1 \end{array} \right] \begin{array}{l} L_2 \leftarrow L_2 - 2L_1 \\ L_3 \leftarrow L_3 - 3L_1 \end{array}$$

Aproveitamos e já indicamos acima as primeiras transformações lineares desejadas, obtendo o seguinte resultado:

$$A^{-1} = \left[\begin{array}{ccc|ccc} 1 & 1 & 2 & 1 & 0 & 0 \\ 0 & 2 & -7 & -2 & 1 & 0 \\ 0 & 3 & -11 & -3 & 0 & 1 \end{array} \right] \quad L_2 \leftarrow \frac{1}{2}L_2$$

Com a indicação da transformação linear necessária para se estabelecer o pivô na segunda linha, obtemos:

$$A^{-1} = \left[\begin{array}{ccc|ccc} 1 & 1 & 2 & 1 & 0 & 0 \\ 0 & 1 & -\frac{7}{2} & -1 & \frac{1}{2} & 0 \\ 0 & 3 & -11 & -3 & 0 & 1 \end{array} \right] \quad \begin{array}{l} L_1 \leftarrow L_1 - L_2 \\ L_3 \leftarrow L_3 - 3L_2 \end{array}$$

A partir das transformações lineares indicadas, teremos:

$$A^{-1} = \left[\begin{array}{ccc|ccc} 1 & 0 & \frac{11}{2} & 2 & -\frac{1}{2} & 0 \\ 0 & 1 & -\frac{7}{2} & -1 & \frac{1}{2} & 0 \\ 0 & 0 & -\frac{1}{2} & 0 & -\frac{3}{2} & 1 \end{array} \right] \quad L_3 \leftarrow -2L_3$$

Com o pivô determinado na terceira linha, estamos quase finalizando o cálculo da inversa, restando as últimas transformações lineares, indicadas abaixo:

$$A^{-1} = \left[\begin{array}{ccc|ccc} 1 & 0 & \frac{11}{2} & 2 & -\frac{1}{2} & 0 \\ 0 & 1 & -\frac{7}{2} & -1 & \frac{1}{2} & 0 \\ 0 & 0 & 1 & 0 & 3 & -2 \end{array} \right] \quad \begin{array}{l} L_1 \leftarrow L_1 - \frac{11}{2}L_3 \\ L_2 \leftarrow L_2 + \frac{7}{2}L_3 \end{array}$$

Após a aplicação das transformações lineares indicadas acima, eliminamos os elementos acima do pivô da terceira coluna, obtendo:

$$A^{-1} = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 2 & -17 & 11 \\ 0 & 1 & 0 & -1 & 11 & -7 \\ 0 & 0 & 1 & 0 & 3 & -2 \end{array} \right]$$

E assim, o lado direito da matriz aumentada é a inversa de A:

$$\mathbf{A}^{-1} = \begin{bmatrix} 2 & -17 & 11 \\ -1 & 11 & -7 \\ 0 & 3 & -2 \end{bmatrix}$$

Com a inversa determinada, podemos calcular a sua norma, que vale:

$$\|\mathbf{A}^{-1}\|_F = \sqrt{2^2 + (-17)^2 + 11^2 + (-1)^2 + (11)^2 + (-7)^2 + 0^2 + 3^2 + (-2)^2} = \sqrt{598} \approx 24,45$$

Logo, podemos calcular o número de condição da matriz com base na norma de Frobenius:

$$\kappa_F(\mathbf{A}) = \|\mathbf{A}\|_F \cdot \|\mathbf{A}^{-1}\|_F = \sqrt{105 \cdot 598} \approx 10,25 \cdot 24,45 \approx 250,58$$

Esse valor elevado¹ indica que, embora o sistema tenha solução única (pois $|\mathbf{A}| = -1 \neq 0$), ele é **mal condicionado**. Isso significa que a solução é sensível a pequenas perturbações nos coeficientes ou nos termos independentes, o que pode comprometer a precisão numérica em implementações computacionais.

Portanto, embora o determinante da matriz não seja pequeno em termos absolutos, e a razão

$$\frac{|\det(\mathbf{A})|}{\|\mathbf{A}\|_F} = \frac{1}{\sqrt{105}} \approx 0,098$$

não seja extremamente próxima de zero, essa métrica isolada **não é adequada** para avaliar a estabilidade numérica do sistema. O número de condição, por outro lado, fornece uma análise mais robusta e confiável. Neste caso, ele indica que o sistema está **mal condicionado**, apesar de ser matematicamente **bem posto** — possui solução única garantida por $\det(\mathbf{A}) \neq 0$ [4].

Embora o sistema possa ser resolvido corretamente por métodos como o de Gauss-Jordan, a presença de um número de condição elevado exige atenção especial à **precisão numérica**. Em implementações computacionais, recomenda-se o uso de **representações com mais casas decimais** ou **aritmética de maior precisão** (como ponto flutuante com 64 ou 128 bits), para mitigar os efeitos de erros de arredondamento que podem ser amplificados. Além disso, é importante validar a sensibilidade da solução com relação às entradas e considerar, quando possível, técnicas de pré-condicionamento ou reformulações do sistema.

¹Valores de número de condição significativamente maiores que 10^2 indicam crescente instabilidade numérica; ou seja, $\kappa \gg 100$ é considerado sinal de forte sensibilidade à entrada.

Revisão do Exemplo 2 – Sistema mal posto e mal condicionado

Retomemos agora o segundo exemplo:

$$\begin{cases} 2x + y = 3 \\ 2x + 1,001y = 0 \end{cases}$$

Com matriz de coeficientes:

$$\mathbf{B} = \begin{bmatrix} 2 & 1 \\ 2 & 1,001 \end{bmatrix}$$

Calculamos anteriormente:

$$|\mathbf{B}| = 0,002; \quad \|\mathbf{B}\|_F = \sqrt{10,002001} \approx 3,16$$

Calculando a inversa de $\mathbf{B}$:

$$\mathbf{B}^{-1} = \frac{1}{0,002} \cdot \begin{bmatrix} 1,001 & -1 \\ -2 & 2 \end{bmatrix} = \begin{bmatrix} 500,5 & -500 \\ -1000 & 1000 \end{bmatrix}$$

A norma de Frobenius da inversa é:

$$\begin{aligned} \|\mathbf{B}^{-1}\|_F &= \sqrt{(500,5)^2 + (-500)^2 + (-1000)^2 + 1000^2} \\ &\approx \sqrt{250500,25 + 250000 + 10^6 + 10^6} \\ &\approx \sqrt{2505500,25} \\ &\approx 1582,22 \end{aligned}$$

Logo, o número de condição é:

$$\kappa(\mathbf{B}) \approx 3,16 \times 1582,22 \approx 5000,2$$

Este valor é extremamente elevado, indicando que o sistema é, de fato, **mal condicionado**.

Logo, ao alterar o coeficiente $1,001 \rightarrow 1,002$, a solução varia de forma significativa:

- Solução original: $x = 1501,5; \quad y = -3000$
- Solução após alteração: $x = 751,5; \quad y = -1500$

Novamente, notem que uma variação de apenas 0,1% em um coeficiente causou uma alteração de 50% na solução — o que evidencia a **sensibilidade extrema** a erros de arredondamento, típica de sistemas mal condicionados.

Diante de um sistema simultaneamente **mal posto e mal condicionado**, como é o caso deste exemplo, o uso de métodos diretos como **Gauss-Jordan** torna-se numericamente arriscado, pois erros de arredondamento podem ser significativamente amplificados. Para lidar com tais situações, recomenda-se a utilização de métodos **mais robustos**, como a **fatoração LU com pivotamento**, que proporciona maior estabilidade numérica. Alternativamente, abordagens iterativas com técnicas de pré-condicionamento também podem ser empregadas, especialmente em sistemas maiores. Além disso, sempre que possível, é aconselhável realizar uma análise do número de condição e reescalar ou reformular o sistema de forma a mitigar a sensibilidade da solução às perturbações nos dados.

Para contrapor toda a discussão teórica que estabelecemos até aqui, vamos analisar um novo exemplo que o sistema é bem posto e bem condicionado.

Exemplo 3 — Sistema bem posto e bem condicionado

Considere o seguinte sistema linear, definido por uma matriz de coeficientes diagonal:

$$\mathbf{C} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix}$$

Como a matriz é diagonal com todos os elementos diferentes de zero, podemos concluir imediatamente que ela é **não singular**, ou seja, possui determinante diferente de zero:

$$|\mathbf{C}| = 2 \cdot 3 \cdot 5 = 30 \neq 0$$

Portanto, o sistema linear associado é **bem posto** e possui **solução única garantida**.

A seguir, analisaremos o **condicionamento numérico** do sistema, utilizando o **número de condição com base na norma de Frobenius**.

Norma de Frobenius da matriz:

$$\|\mathbf{C}\|_F = \sqrt{2^2 + 3^2 + 5^2} = \sqrt{4 + 9 + 25} = \sqrt{38} \approx 6,16$$

A matriz inversa de $\mathbf{C}$ também é diagonal:

$$\mathbf{C}^{-1} = \begin{bmatrix} \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{5} \end{bmatrix}$$

Norma de Frobenius da inversa:

$$\|\mathbf{C}^{-1}\|_F = \sqrt{\left(\frac{1}{2}\right)^2 + \left(\frac{1}{3}\right)^2 + \left(\frac{1}{5}\right)^2} = \sqrt{\frac{1}{4} + \frac{1}{9} + \frac{1}{25}} = \sqrt{\frac{1133}{9000}} \approx 0,355$$

Número de condição na norma de Frobenius:

$$\kappa_F(\mathbf{C}) = \|\mathbf{C}\|_F \cdot \|\mathbf{C}^{-1}\|_F \approx 6,16 \cdot 0,355 \approx 2,19$$

Esse valor é **muito próximo de 1** — consequentemente, bem distante de zero — o que caracteriza tal sistema como **bem condicionado**, ou seja, numericamente estável. A solução desse sistema será pouco sensível a pequenas variações nos coeficientes ou nos termos independentes, mesmo em ambientes computacionais com precisão finita.

Por fim, embora a métrica da razão entre determinante e norma de Frobenius seja apenas uma estimativa grosseira da “robustez” numérica da matriz, vale a pena observá-la para efeito comparativo:

$$\frac{||\mathbf{C}||}{\|\mathbf{C}\|_F} = \frac{30}{\sqrt{38}} \approx 4,86$$

Esse valor, por ser significativamente maior do que 1, sugere que o determinante não é pequeno em relação à escala geral da matriz – reforçando, de forma complementar, a percepção de que o sistema não apresenta riscos numéricos imediatos. No entanto, é importante destacar que essa razão **não substitui** o número de condição, que é a medida adequada para avaliar a sensibilidade do sistema.

Conclusão: Os exemplos apresentados demonstram que a análise de estabilidade numérica de sistemas lineares não pode se limitar à verificação da existência e unicidade da solução (isto é, à análise do posto ou do determinante da matriz). Um sistema pode ser **bem posto** – ou seja, possuir solução única garantida – e, ainda assim, ser **numericamente instável**, como evidenciado no primeiro exemplo.

O **Exemplo 1**, apesar de bem posto e ter determinante igual a -1 , revelou um **número de condição elevado**, indicando forte sensibilidade a perturbações – ou seja, **mal condicionado**. O **Exemplo 2**, mal posto e com determinante pequeno ($0,002$), exibiu ainda maior instabilidade, sendo um caso clássico de **sistema mal condicionado**, onde pequenas variações nos dados causam grandes erros na solução.

Por outro lado, o **Exemplo 3** apresentou uma matriz diagonal com número de condição próximo de 1, revelando um sistema **bem posto e bem condicionado**, no qual a solução é estável mesmo sob erros de arredondamento ou pequenas perturbações nos dados.

Dessa forma, fica evidente que:

- A não singularidade (isto é, $\det(\mathbf{A}) \neq 0$) **garante existência e unicidade**, mas não assegura estabilidade;
- A magnitude do determinante, isoladamente, **não é um critério confiável** de estabilidade numérica;
- O **número de condição** é a ferramenta adequada para quantificar a sensibilidade do sistema a perturbações e, portanto, deve ser sempre considerado em aplicações

computacionais.

Em implementações numéricas, o uso de métodos robustos e bem analisados — como decomposição LU com pivotamento, métodos iterativos adaptativos ou técnicas de regularização — torna-se essencial para tratar sistemas mal condicionados e minimizar os efeitos da instabilidade.

“A precisão da solução não depende apenas do método, mas da natureza do problema — e da atenção do engenheiro ao reconhecê-la.”

Lista de Exercícios

2.1 Encontre as soluções dos seguintes sistemas (se existirem), de forma analítica e computacional, utilizando o método de Gauss-Jordan. Caso não existam soluções, apresente em detalhes a justificativa!

(a)

$$\begin{cases} 2x + 3y = 5 \\ 4x - y = 7 \end{cases}$$

Gabarito:

Matriz aumentada:

$$\left[\begin{array}{cc|c} 2 & 3 & 5 \\ 4 & -1 & 7 \end{array} \right]$$

$$L_1 \leftarrow \frac{1}{2}L_1 \Rightarrow \left[\begin{array}{cc|c} 1 & \frac{3}{2} & \frac{5}{2} \\ 4 & -1 & 7 \end{array} \right]$$

$$L_2 \leftarrow L_2 - 4L_1 \Rightarrow \left[\begin{array}{cc|c} 1 & \frac{3}{2} & \frac{5}{2} \\ 0 & -7 & -3 \end{array} \right]$$

$$L_2 \leftarrow -\frac{1}{7}L_2 \Rightarrow \left[\begin{array}{cc|c} 1 & \frac{3}{2} & \frac{5}{2} \\ 0 & 1 & \frac{3}{7} \end{array} \right]$$

$$L_1 \leftarrow L_1 - \frac{3}{2}L_2 \Rightarrow \left[\begin{array}{cc|c} 1 & 0 & \frac{13}{7} \\ 0 & 1 & \frac{3}{7} \end{array} \right]$$

Solução única:

$$\boxed{x = \frac{13}{7}, \quad y = \frac{3}{7}}$$

<

(b)

$$\begin{cases} 2x + 3y + z = 0 \\ x - 2y - z = 1 \\ x + 4y + z = 2 \end{cases}$$

Gabarito:

Matriz aumentada:

$$\left[\begin{array}{ccc|c} 2 & 3 & 1 & 0 \\ 1 & -2 & -1 & 1 \\ 1 & 4 & 1 & 2 \end{array} \right]$$

Após as operações elementares de Gauss-Jordan (normalização dos pivôs e eliminação acima e abaixo de cada pivô) chega-se à forma RREF:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & -\frac{1}{4} \\ 0 & 1 & 0 & \frac{7}{4} \\ 0 & 0 & 1 & -\frac{19}{4} \end{array} \right]$$

Solução única:

$$x = -\frac{1}{4}, \quad y = \frac{7}{4}, \quad z = -\frac{19}{4}$$

<

(c)

$$\begin{cases} 2x + 2y - 3z = 4 \\ x + 3y + z = 11 \\ 2x + 5y - 4z = 13 \end{cases}$$

Gabarito:

Matriz aumentada:

$$\left[\begin{array}{ccc|c} 2 & 2 & -3 & 4 \\ 1 & 3 & 1 & 11 \\ 2 & 5 & -4 & 13 \end{array} \right]$$

Após Gauss-Jordan obtém-se a RREF:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & \frac{2}{19} \\ 0 & 1 & 0 & \frac{63}{19} \\ 0 & 0 & 1 & \frac{18}{19} \end{array} \right]$$

Solução única:

$$x = \frac{2}{19}, \quad y = \frac{63}{19}, \quad z = \frac{18}{19}$$

<

(d)

$$\begin{cases} x + y + z = 1 \\ x - y + z = 2 \\ 2x + 2y + 2z = 5 \end{cases}$$

Gabarito:

Matriz aumentada:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 2 \\ 2 & 2 & 2 & 5 \end{array} \right]$$

$$L_2 \leftarrow L_2 - L_1, \quad L_3 \leftarrow L_3 - 2L_1 \Rightarrow \left[\begin{array}{ccc|c} 1 & 1 & 1 & 1 \\ 0 & -2 & 0 & 1 \\ 0 & 0 & 0 & 3 \end{array} \right]$$

A última linha é $0 = 3$, contradição. O posto da matriz de coeficientes é 2 e o posto da matriz aumentada é 3.

Sistema inconsistente: não possui solução.

<

(e)

$$\begin{cases} x + y + z = 1 \\ x - y + z = 2 \\ 2x + 2z = 3 \end{cases}$$

Gabarito:

Matriz aumentada:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 2 \\ 2 & 0 & 2 & 3 \end{array} \right]$$

$$L_2 \leftarrow L_2 - L_1, \quad L_3 \leftarrow L_3 - 2L_1 \Rightarrow \left[\begin{array}{ccc|c} 1 & 1 & 1 & 1 \\ 0 & -2 & 0 & 1 \\ 0 & -2 & 0 & 1 \end{array} \right]$$

$$L_3 \leftarrow L_3 - L_2 \Rightarrow \left[\begin{array}{ccc|c} 1 & 1 & 1 & 1 \\ 0 & -2 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{array} \right]$$

$$L_2 \leftarrow -\frac{1}{2}L_2, \quad L_1 \leftarrow L_1 - L_2 \Rightarrow \left[\begin{array}{ccc|c} 1 & 0 & 1 & \frac{3}{2} \\ 0 & 1 & 0 & -\frac{1}{2} \\ 0 & 0 & 0 & 0 \end{array} \right]$$

Há uma variável livre ($z = t$).

Infinitas soluções:

$$x = \frac{3}{2} - t, \quad y = -\frac{1}{2}, \quad z = t, \quad t \in \mathbb{R}.$$

<

(f)

$$\begin{cases} 2x + y - 2z = 10 \\ 3x + 2y + 2z = 1 \\ 5x + 4y + 3z = 4 \end{cases}$$

Gabarito:

Matriz aumentada:

$$\left[\begin{array}{ccc|c} 2 & 1 & -2 & 10 \\ 3 & 2 & 2 & 1 \\ 5 & 4 & 3 & 4 \end{array} \right]$$

Após Gauss-Jordan obtém-se a RREF:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & -3 \end{array} \right]$$

Solução única:

$$x = 1, \quad y = 2, \quad z = -3$$

<

2.2 Determine se a seguinte matriz é singular:

$$A = \begin{bmatrix} 2.1 & -0.6 & 1.1 \\ 3.2 & 4.7 & -0.8 \\ 3.1 & -6.5 & 4.1 \end{bmatrix}$$

Gabarito:

Para verificar se a matriz A é singular, calculamos o seu determinante pelo método de eliminação de Gauss (sem normalização dos pivôs), conforme a propriedade

$$\det(A) = (-1)^s \prod_{i=1}^n a_{ii},$$

onde s é o número de trocas de linha (neste caso $s = 0$).

Matriz inicial:

$$A = \begin{bmatrix} 2,1 & -0,6 & 1,1 \\ 3,2 & 4,7 & -0,8 \\ 3,1 & -6,5 & 4,1 \end{bmatrix}$$

1º passo – Eliminação da primeira coluna:

$$\begin{matrix} L_2 \leftarrow L_2 - \frac{3,2}{2,1} L_1 \\ L_3 \leftarrow L_3 - \frac{3,1}{2,1} L_1 \end{matrix} \Rightarrow \begin{bmatrix} 2,1 & -0,6 & 1,1 \\ 0 & 5,61428571 & -2,47619048 \\ 0 & -5,61428571 & 2,47619048 \end{bmatrix}$$

2º passo – Eliminação da segunda coluna:

$$L_3 \leftarrow L_3 - \left(\frac{-5,61428571}{5,61428571} \right) L_2 = L_3 + L_2 \Rightarrow \begin{bmatrix} 2,1 & -0,6 & 1,1 \\ 0 & 5,61428571 & -2,47619048 \\ 0 & 0 & \approx 0 \end{bmatrix}$$

O elemento a_{33} resultante é numericamente nulo ($\approx -8,88 \times 10^{-16}$, erro de arredondamento de ponto flutuante). Assim,

$$\det(A) = 2,1 \cdot 5,61428571 \cdot 0 = 0.$$

Como $\det(A) = 0$, a matriz A é **singular**.

Além disso, o posto de A é 2 (inferior à ordem da matriz), confirmando a singularidade.



2.3 Verifique se o sistema a seguir é **não singular**. Caso afirmativo, resolva-o pelo método de Gauss-Jordan, considerando a equação $A\mathbf{x} = \mathbf{b}$, onde:

$$A = \begin{bmatrix} 8 & -6 & 2 \\ -4 & 11 & -7 \\ 4 & -7 & 6 \end{bmatrix} \quad \text{e} \quad B = \begin{bmatrix} 28 \\ -40 \\ 33 \end{bmatrix}$$

Gabarito:

1. Verificação da não singularidade

Calculamos o determinante de A pelo método de eliminação de Gauss (sem normalização dos pivôs):

$$\det(A) = \det \left(\begin{bmatrix} 8 & -6 & 2 \\ -4 & 11 & -7 \\ 4 & -7 & 6 \end{bmatrix} \right) \begin{matrix} L_2 \leftarrow L_2 + \frac{1}{2}L_1 \\ L_3 \leftarrow L_3 - \frac{1}{2}L_1 \end{matrix} = \begin{vmatrix} 8 & -6 & 2 \\ 0 & 8 & -6 \\ 0 & -4 & 5 \end{vmatrix}$$

$$L_3 \leftarrow L_3 + \frac{1}{2}L_2 \Rightarrow \begin{vmatrix} 8 & -6 & 2 \\ 0 & 8 & -6 \\ 0 & 0 & 2 \end{vmatrix}$$

$$\det(A) = 8 \cdot 8 \cdot 2 = 128 \neq 0.$$

Como $\det(A) \neq 0$, a matriz A é **não singular** e o sistema possui solução única.

2. Resolução pelo método de Gauss-Jordan

Matriz aumentada $[A \mid \mathbf{b}]$:

$$\left[\begin{array}{ccc|c} 8 & -6 & 2 & 28 \\ -4 & 11 & -7 & -40 \\ 4 & -7 & 6 & 33 \end{array} \right]$$

$$L_1 \leftarrow \frac{1}{8}L_1 \Rightarrow \left[\begin{array}{ccc|c} 1 & -\frac{3}{4} & \frac{1}{4} & \frac{7}{2} \\ -4 & 11 & -7 & -40 \\ 4 & -7 & 6 & 33 \end{array} \right]$$

$$L_2 \leftarrow L_2 + 4L_1, \quad L_3 \leftarrow L_3 - 4L_1 \Rightarrow \left[\begin{array}{ccc|c} 1 & -\frac{3}{4} & \frac{1}{4} & \frac{7}{2} \\ 0 & 8 & -6 & -26 \\ 0 & -4 & 5 & 19 \end{array} \right]$$

$$L_2 \leftarrow \frac{1}{8}L_2 \Rightarrow \left[\begin{array}{ccc|c} 1 & -\frac{3}{4} & \frac{1}{4} & \frac{7}{2} \\ 0 & 1 & -\frac{3}{4} & -\frac{13}{4} \\ 0 & -4 & 5 & 19 \end{array} \right]$$

$$L_1 \leftarrow L_1 + \frac{3}{4}L_2, \quad L_3 \leftarrow L_3 + 4L_2 \Rightarrow \left[\begin{array}{ccc|c} 1 & 0 & -\frac{5}{16} & \frac{17}{16} \\ 0 & 1 & -\frac{3}{4} & -\frac{13}{4} \\ 0 & 0 & 2 & 6 \end{array} \right]$$

$$L_3 \leftarrow \frac{1}{2}L_3 \Rightarrow \left[\begin{array}{ccc|c} 1 & 0 & -\frac{5}{16} & \frac{17}{16} \\ 0 & 1 & -\frac{3}{4} & -\frac{13}{4} \\ 0 & 0 & 1 & 3 \end{array} \right]$$

$$L_1 \leftarrow L_1 + \frac{5}{16}L_3, \quad L_2 \leftarrow L_2 + \frac{3}{4}L_3 \Rightarrow \left[\begin{array}{ccc|c} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 3 \end{array} \right]$$

Solução única:

$$x = 2, \quad y = -1, \quad z = 3$$



2.4 Mariana comprou 2 cadernos e 3 canetas por R\$ 24,00. No dia seguinte, comprou 1 caderno e 2 canetas da mesma marca por R\$ 15,00. Através do Método de Gauss-Jordan, determine qual o preço de um caderno e de uma caneta.

Gabarito:

Sejam:

c = preço de um caderno, p = preço de uma caneta.

As compras realizadas fornecem o sistema linear:

$$\begin{cases} 2c + 3p = 24, \\ c + 2p = 15. \end{cases}$$

Podemos escrever esse sistema na forma matricial aumentada:

$$\left[\begin{array}{cc|c} 2 & 3 & 24 \\ 1 & 2 & 15 \end{array} \right].$$

1º passo: Trocamos as linhas para obter um pivô igual a 1 na primeira linha:

$$L_1 \leftrightarrow L_2$$

$$\left[\begin{array}{cc|c} 1 & 2 & 15 \\ 2 & 3 & 24 \end{array} \right].$$

2º passo: Zerar o elemento abaixo do pivô da primeira coluna:

$$L_2 \leftarrow L_2 - 2L_1,$$

resultando em:

$$\left[\begin{array}{cc|c} 1 & 2 & 15 \\ 0 & -1 & -6 \end{array} \right].$$

3º passo: Tornar o pivô da segunda linha igual a 1:

$$L_2 \leftarrow -1L_2,$$

$$\left[\begin{array}{cc|c} 1 & 2 & 15 \\ 0 & 1 & 6 \end{array} \right].$$

4º passo: Zerar o elemento acima do pivô da segunda linha:

$$L_1 \leftarrow L_1 - 2L_2,$$

$$\left[\begin{array}{cc|c} 1 & 0 & 3 \\ 0 & 1 & 6 \end{array} \right].$$

Assim, obtemos a solução:

$c = 3, \qquad p = 6.$

Portanto:

Um caderno custa R\$ 3,00,

Uma caneta custa R\$ 6,00.

2.5 Um fazendeiro precisa misturar três rações (A, B e C) para fornecer quantidades específicas de proteína, gordura e fibra. As porcentagens de nutrientes em cada ração são dadas pela tabela abaixo, em relação a cada quilograma de ração:

Nutrientes	Ração A	Ração B	Ração C
Proteína	10%	8%	6%
Gordura	6%	7%	5%
Fibras	5%	6%	4%

O fazendeiro deseja atingir 68g de proteína, 53g de gordura e 44g de fibra, sendo que cada kg de ração contém a respectiva porcentagem. Com base nessa informação, podemos montar o seguinte sistema de equações lineares:

$$\left\{ \begin{array}{l} 10x + 8y + 6z = 68 \\ 6x + 7y + 5z = 53 \\ 5x + 6y + 4z = 44 \end{array} \right. \qquad (2.8)$$

onde x é a quantidade da ração A, y a quantidade da ração B e z a quantidade da ração C – todas as medidas em quilogramas.

(A) Demonstre que o sistema não é singular.

Para demonstrar a não singularidade do sistema (e que, portanto, existe apenas uma única solução), basta calcular o determinante da matriz de coeficientes e verificar que o seu valor é diferente de zero!

Passos do método da Eliminação de Gauss no cálculo do Determinante da

matriz de coeficientes:

$$\begin{aligned} \det(A) &= \det \begin{pmatrix} 10 & 8 & 6 \\ 6 & 7 & 5 \\ 5 & 6 & 4 \end{pmatrix} \quad \begin{array}{l} L_2 \leftarrow L_2 - \frac{6}{10}L_1 \\ L_3 \leftarrow L_3 - \frac{5}{10}L_1 \end{array} \\ &= \begin{vmatrix} 10 & 8 & 6 \\ 0 & 2,2 & 1,4 \\ 0 & 2,0 & 1,0 \end{vmatrix} \quad L_3 \leftarrow L_3 - \frac{2,0}{2,2}L_2 \\ &= \begin{vmatrix} 10 & 8 & 6 \\ 0 & 2,2 & 1,4 \\ 0 & 0 & -0,272 \end{vmatrix} \\ \det(A) &= 10 * 2,2 * (-0,272) \cong -6,000 \end{aligned}$$

Logo, o sistema não é singular.

(B) Calcule sua **norma de Frobenius**, definida por:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n |a_{ij}|^2},$$

De acordo com a equação da **norma de Frobenius**, temos:

$$\begin{aligned} \|\mathbf{A}\|_F &= \sqrt{10^2 + 8^2 + 6^2 + 6^2 + 7^2 + 5^2 + 5^2 + 6^2 + 4^2} \\ &= \sqrt{100 + 64 + 36 + 36 + 49 + 25 + 25 + 36 + 16} \\ &= \sqrt{387} \\ \|\mathbf{A}\|_F &= 19,6723 \end{aligned}$$

(C) Determine seu **número condição**², definido como:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$$

onde $\|\cdot\|$ representa uma norma matricial.

Para determinar o **número condição** do sistema, primeiro precisamos calcular a inversa da matriz de coeficientes A , através do método de Gauss-Jordan, conforme determinado pelo enunciado. O processo exige que construamos a

²O cálculo da inversa também deverá ser realizado através do método de Gauss-Jordan.

matriz aumentada na forma $[A|I]$, de modo que ao final da aplicação do método teremos $[I|A^{-1}]$.

Detalhamento dos cálculos da inversa de A :

$$\begin{aligned}
 \text{Aug} &= \left[\begin{array}{ccc|ccc} 10 & 8 & 6 & 1 & 0 & 0 \\ 6 & 7 & 5 & 0 & 1 & 0 \\ 5 & 6 & 4 & 0 & 0 & 1 \end{array} \right] \quad L_1 \leftarrow L_1 - \frac{L_1}{10} \\
 &= \left[\begin{array}{ccc|ccc} 1 & 0,8 & 0,6 & 0,1 & 0 & 0 \\ 6 & 7 & 5 & 0 & 1 & 0 \\ 5 & 6 & 4 & 0 & 0 & 1 \end{array} \right] \quad \begin{array}{l} L_2 \leftarrow L_2 - 6L_1 \\ L_1 \leftarrow L_3 - 5L_1 \end{array} \\
 &= \left[\begin{array}{ccc|ccc} 1 & 0,8 & 0,6 & 0,1 & 0 & 0 \\ 0 & 2,2 & 1,4 & -0,6 & 1 & 0 \\ 0 & 2,0 & 1,0 & -0,5 & 0 & 1 \end{array} \right] \quad L_2 \leftarrow \frac{L_2}{2,2} \\
 \text{Aug} &= \left[\begin{array}{ccc|ccc} 1 & 0,8 & 0,6 & 0,1 & 0 & 0 \\ 0 & 1 & 0,63636 & -0,2727 & 0,4545 & 0 \\ 0 & 2,0 & 1,0 & -0,5 & 0 & 1 \end{array} \right] \quad \begin{array}{l} L_1 \leftarrow L_1 - 0,8L_2 \\ L_3 \leftarrow L_3 - 2,0L_2 \end{array} \\
 &= \left[\begin{array}{ccc|ccc} 1 & 0 & 0,0909 & 0,31818 & -0,3636 & 0 \\ 0 & 1 & 0,63636 & -0,2727 & 0,4545 & 0 \\ 0 & 0 & -0,2727 & 0,04545 & -0,90909 & 1 \end{array} \right] \quad L_3 \leftarrow -\frac{L_3}{0,2727} \\
 &= \left[\begin{array}{ccc|ccc} 1 & 0 & 0,0909 & 0,31818 & -0,3636 & 0 \\ 0 & 1 & 0,6363 & -0,2727 & 0,4545 & 0 \\ 0 & 0 & 1 & -0,1667 & 3,3333 & -3,6667 \end{array} \right] \quad \begin{array}{l} L_1 \leftarrow L_1 - 0,0909L_3 \\ L_2 \leftarrow L_2 - 0,6363L_3 \end{array} \\
 \text{Aug} &= \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0,3333 & -0,6667 & 0,3333 \\ 0 & 1 & 0 & -0,1667 & -1,6667 & 2,3333 \\ 0 & 0 & 1 & -0,1667 & 3,3333 & -3,6667 \end{array} \right]
 \end{aligned}$$

ou seja, a matriz inversa é:

$$A^{-1} = \begin{bmatrix} 0,3333 & -0,6667 & 0,3333 \\ -0,1667 & -1,6667 & 2,3333 \\ -0,1667 & 3,3333 & -3,6667 \end{bmatrix}$$

Agora, precisamos calcular a **norma de Frobenius** dessa matriz. O procedimento é o mesmo que aplicamos na matriz A :

$$\begin{aligned}
\|\mathbf{A}^{-1}\|_F &= \sqrt{0,3333^2 + (-0,6667)^2 + 0,3333^2 + (-0,1667)^2 + (-1,667)^2 + 2,3333^2 + (-0,1667)^2 + 3,3333^2 + (-3,6667)^2} \\
&= \sqrt{0,1111 + 0,4445 + 0,1111 + 0,0278 + 2,7789 + 5,4443 + 0,0278 + 11,1109 + 13,4445} \\
&= \sqrt{33,5009} \\
\|\mathbf{A}^{-1}\|_F &\cong 5,7871
\end{aligned}$$

Portanto, de acordo com a definição matemática do **número condição**, temos:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| = (19,6723) \cdot (5,7871) = 113,86.$$

Esse valor indica que o sistema apresenta sensibilidade numérica e que, portanto, devemos ter cuidado com a precisão e a representação dos números calculados.

(D) Resolva o sistema através método de Gauss-Jordan.

Aplicando o método de Gauss-Jordan na resolução do sistema linear.

Detalhamento dos passos:

$$\begin{aligned}
 Aug &= \left[\begin{array}{ccc|c} 10 & 8 & 6 & 68 \\ 6 & 7 & 5 & 53 \\ 5 & 6 & 4 & 44 \end{array} \right] \quad L_1 \leftarrow L_1 - \frac{L_1}{10} \\
 &= \left[\begin{array}{ccc|c} 1,0 & 0,8 & 0,6 & 6,8 \\ 6 & 7 & 5 & 53 \\ 5 & 6 & 4 & 44 \end{array} \right] \quad \begin{array}{l} L_2 \leftarrow L_2 - 6L_1 \\ L_3 \leftarrow L_3 - 5L_1 \end{array} \\
 &= \left[\begin{array}{ccc|c} 1,0 & 0,8 & 0,6 & 6,8 \\ 0 & 2,2 & 1,4 & 12,2 \\ 0 & 2,0 & 1,0 & 10,0 \end{array} \right] \quad L_2 \leftarrow \frac{L_2}{2,2} \\
 &= \left[\begin{array}{ccc|c} 1,0 & 0,8 & 0,6 & 6,8 \\ 0 & 1,0 & 0,6363 & 5,54545 \\ 0 & 2,0 & 1,0 & 10,0 \end{array} \right] \quad \begin{array}{l} L_1 \leftarrow L_1 - 0,8L_2 \\ L_3 \leftarrow L_3 - 2,0L_2 \end{array} \\
 &= \left[\begin{array}{ccc|c} 1,0 & 0 & 0,0909 & 2,3636 \\ 0 & 1,0 & 0,6363 & 5,54545 \\ 0 & 0 & -0,2727 & -1,0909 \end{array} \right] \quad L_3 \leftarrow -\frac{L_3}{0,2727} \\
 &= \left[\begin{array}{ccc|c} 1,0 & 0 & 0,0909 & 2,3636 \\ 0 & 1,0 & 0,6363 & 5,54545 \\ 0 & 0 & 1,0 & 4,0 \end{array} \right] \quad \begin{array}{l} L_1 \leftarrow L_1 - 0,0909L_3 \\ L_2 \leftarrow L_2 - 0,6363L_3 \end{array} \\
 Aug &= \left[\begin{array}{ccc|c} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 4 \end{array} \right]
 \end{aligned}$$

Portanto, a **solução** do sistema linear é: $x = 2$, $y = 3$, $z = 4$. Ou seja, misturando-se 2 kg da ração A, 3 kg da ração B e 4 kg da ração C as proporções de proteína, gordura e fibra serão atendidas.

2.6 Uma fábrica possui 3 máquinas (X, Y, Z) que produzem peças por hora, conforme a tabela:

Máquina	Tipo A	Tipo B	Tipo C
X	3	2	1
Y	2	3	2
Z	1	2	3

Para atender à demanda de 34 unidades do tipo A, 38 unidades do tipo B e 30 unidades do tipo C, precisamos analisar o seguinte sistema linear:

$$\begin{cases} 3x + 2y + z = 34 \\ 2x + 3y + 2z = 38 \\ x + 2y + 3z = 30 \end{cases} \quad (2.9)$$

(A) Demonstre que o sistema não é singular.

Gabarito:

Para demonstrar que o sistema não é singular, basta calcular o determinante da matriz de coeficientes:

$$A = \begin{bmatrix} 3 & 2 & 1 \\ 2 & 3 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

Aplicando a eliminação de Gauss para calcular o determinante:

$$\det(A) = \begin{vmatrix} 3 & 2 & 1 \\ 2 & 3 & 2 \\ 1 & 2 & 3 \end{vmatrix} \begin{array}{l} L_2 \leftarrow L_2 - \frac{2}{3}L_1 \\ L_3 \leftarrow L_3 - \frac{1}{3}L_1 \end{array}$$

$$= \begin{vmatrix} 3 & 2 & 1 \\ 0 & \frac{5}{3} & \frac{4}{3} \\ 0 & \frac{4}{3} & \frac{8}{3} \end{vmatrix} L_3 \leftarrow L_3 - \frac{4}{5}L_2$$

$$= \begin{vmatrix} 3 & 2 & 1 \\ 0 & \frac{5}{3} & \frac{4}{3} \\ 0 & 0 & \frac{8}{5} \end{vmatrix}$$

$$\det(A) = 3 \cdot \frac{5}{3} \cdot \frac{8}{5} = 8 \neq 0.$$

Logo, o sistema é **não singular**.

(B) Calcule sua **norma de Frobenius**, definida por:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n |a_{ij}|^2},$$

onde $\|\cdot\|$ representa uma norma matricial. **Gabarito:**

A **norma de Frobenius** é dada por

$$\|A\|_F = \sqrt{3^2 + 2^2 + 1^2 + 2^2 + 3^2 + 2^2 + 1^2 + 2^2 + 3^2}.$$

$$= \sqrt{9 + 4 + 1 + 4 + 9 + 4 + 1 + 4 + 9} = \sqrt{45}$$

$$\|A\|_F = 6,7082$$

(C) Determine seu **número condição**³, definido como:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$$

Gabarito:

Para calcular o número condição, primeiro encontramos a inversa da matriz A usando o método de Gauss–Jordan.

$$\text{Aug} = \left[\begin{array}{ccc|ccc} 3 & 2 & 1 & 1 & 0 & 0 \\ 2 & 3 & 2 & 0 & 1 & 0 \\ 1 & 2 & 3 & 0 & 0 & 1 \end{array} \right]$$

Passos do método:

$$L_1 \leftarrow \frac{1}{3}L_1$$

$$\left[\begin{array}{ccc|ccc} 1 & \frac{2}{3} & \frac{1}{3} & \frac{1}{3} & 0 & 0 \\ 2 & 3 & 2 & 0 & 1 & 0 \\ 1 & 2 & 3 & 0 & 0 & 1 \end{array} \right]$$

$$L_2 \leftarrow L_2 - 2L_1, \quad L_3 \leftarrow L_3 - L_1$$

$$\left[\begin{array}{ccc|ccc} 1 & \frac{2}{3} & \frac{1}{3} & \frac{1}{3} & 0 & 0 \\ 0 & \frac{5}{3} & \frac{4}{3} & -\frac{2}{3} & 1 & 0 \\ 0 & \frac{4}{3} & \frac{8}{3} & -\frac{1}{3} & 0 & 1 \end{array} \right]$$

³O cálculo da inversa também deverá ser realizado através do método de Gauss-Jordan.

$$L_2 \leftarrow \frac{3}{5}L_2$$

$$\left[\begin{array}{ccc|ccc} 1 & \frac{2}{3} & \frac{1}{3} & \frac{1}{3} & 0 & 0 \\ 0 & 1 & \frac{4}{5} & -\frac{2}{5} & \frac{3}{5} & 0 \\ 0 & \frac{4}{3} & \frac{8}{3} & -\frac{1}{3} & 0 & 1 \end{array} \right]$$

$$L_1 \leftarrow L_1 - \frac{2}{3}L_2, \quad L_3 \leftarrow L_3 - \frac{4}{3}L_2$$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & -\frac{1}{5} & \frac{3}{5} & -\frac{2}{5} & 0 \\ 0 & 1 & \frac{4}{5} & -\frac{2}{5} & \frac{3}{5} & 0 \\ 0 & 0 & \frac{8}{5} & \frac{1}{5} & -\frac{4}{5} & 1 \end{array} \right]$$

$$L_3 \leftarrow \frac{5}{8}L_3$$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & -\frac{1}{5} & \frac{3}{5} & -\frac{2}{5} & 0 \\ 0 & 1 & \frac{4}{5} & -\frac{2}{5} & \frac{3}{5} & 0 \\ 0 & 0 & 1 & \frac{1}{8} & -\frac{1}{2} & \frac{5}{8} \end{array} \right]$$

$$L_1 \leftarrow L_1 + \frac{1}{5}L_3, \quad L_2 \leftarrow L_2 - \frac{4}{5}L_3$$

$$\text{Aug} = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{5}{8} & -\frac{1}{2} & \frac{1}{8} \\ 0 & 1 & 0 & -\frac{1}{2} & 1 & -\frac{1}{2} \\ 0 & 0 & 1 & \frac{1}{8} & -\frac{1}{2} & \frac{5}{8} \end{array} \right]$$

Assim:

$$A^{-1} = \begin{bmatrix} \frac{5}{8} & -\frac{1}{2} & \frac{1}{8} \\ -\frac{1}{2} & 1 & -\frac{1}{2} \\ \frac{1}{8} & -\frac{1}{2} & \frac{5}{8} \end{bmatrix}$$

Agora, calculamos sua norma de Frobenius:

$$\begin{aligned} \|A^{-1}\|_F &= \sqrt{\left(\frac{5}{8}\right)^2 + \left(-\frac{1}{2}\right)^2 + \left(\frac{1}{8}\right)^2 + \left(-\frac{1}{2}\right)^2 + 1^2 + \left(-\frac{1}{2}\right)^2 + \left(\frac{1}{8}\right)^2 + \left(-\frac{1}{2}\right)^2 + \left(\frac{5}{8}\right)^2} \\ &= \sqrt{2.8125} \approx 1.67705 \end{aligned}$$

Número condição:

$$\kappa(A) = \|A\|_F \cdot \|A^{-1}\|_F = (6.7082)(1.67705) \approx 11.25$$

$$\kappa(A) \approx 11.25$$

- (D) Resolva o sistema através do método de Gauss–Jordan e determine quantas horas cada uma das máquinas precisa trabalhar para conseguir entregar os pedidos demandados.

Gabarito:

Montamos a matriz aumentada correspondente:

$$\begin{aligned} \text{Aug} &= \left[\begin{array}{ccc|c} 3 & 2 & 1 & 34 \\ 2 & 3 & 2 & 38 \\ 1 & 2 & 3 & 30 \end{array} \right] \begin{array}{l} L_1 \leftarrow L_3 \\ L_2 \leftarrow L_2 - 2L_1 \\ L_3 \leftarrow L_1 \end{array} = \left[\begin{array}{ccc|c} 1 & 2 & 3 & 30 \\ 2 & 3 & 2 & 38 \\ 3 & 2 & 1 & 34 \end{array} \right] \begin{array}{l} L_2 \leftarrow L_2 - 2L_1 \\ L_3 \leftarrow L_3 - 3L_1 \end{array} \\ &= \left[\begin{array}{ccc|c} 1 & 2 & 3 & 30 \\ 0 & -1 & -4 & -22 \\ 0 & -4 & -8 & -56 \end{array} \right] \begin{array}{l} L_2 \leftarrow L_2 \cdot (-1) \end{array} \end{aligned}$$

$$\begin{aligned} \text{Aug} &= \left[\begin{array}{ccc|c} 1 & 2 & 3 & 30 \\ 0 & 1 & 4 & 22 \\ 0 & -4 & -8 & -56 \end{array} \right] \begin{array}{l} L_1 \leftarrow L_1 - 2L_2 \\ L_3 \leftarrow L_3 + 4L_2 \end{array} \\ &= \left[\begin{array}{ccc|c} 1 & 0 & -5 & -14 \\ 0 & 1 & 4 & 22 \\ 0 & 0 & 8 & 32 \end{array} \right] \begin{array}{l} L_1 \leftarrow L_1 + 5L_3 \\ L_2 \leftarrow L_2 - 4L_3 \\ L_3 \leftarrow \frac{L_3}{8} \end{array} = \left[\begin{array}{ccc|c} 1 & 0 & -5 & -14 \\ 0 & 1 & 4 & 22 \\ 0 & 0 & 1 & 4 \end{array} \right] \begin{array}{l} L_1 \leftarrow L_1 + 5L_3 \\ L_2 \leftarrow L_2 - 4L_3 \end{array} \end{aligned}$$

ou seja:

$$\text{Aug} = \left[\begin{array}{ccc|c} 1 & 0 & 0 & 6 \\ 0 & 1 & 0 & 6 \\ 0 & 0 & 1 & 4 \end{array} \right]$$

Portanto, as quantidades de horas necessárias de trabalho por máquina são:

$$x = 6 \text{ horas}, \quad y = 6 \text{ horas}, \quad z = 4 \text{ horas}.$$

Fundamentação da Implementação Numérica

A implementação apresentada na classe `LSSolver` visa resolver sistemas lineares utilizando os métodos diretos de Eliminação de Gauss e Gauss-Jordan, além de fornecer métricas de estabilidade e bem-postura do sistema, como o determinante, número de condição e norma de Frobenius.

```
Projeto/
|
+-- gauss_01.py
+-- LS_Lib/
|   +-- solver.py
```

Figura 2.1: Estrutura de diretórios do projeto

Cada método da classe está fundamentado em princípios teóricos clássicos da Álgebra Linear Numérica ou das melhores práticas computacionais, conforme detalhado a seguir:

- **A estrutura da classe** `LSSolver` foi concebida com foco em modularidade e reaproveitamento dos dados de entrada. O método construtor:

```
1 def __init__(self, augMat):
2     self.augMat = augMat
```

tem como propósito armazenar internamente a matriz aumentada do sistema linear fornecido, representada por uma lista de listas em Python. A matriz aumentada, usualmente denotada por $\tilde{A} = [A | b]$, incorpora tanto os coeficientes do sistema A quanto o vetor dos termos independentes b , permitindo que os métodos da classe operem diretamente sobre ela sem necessidade de repassar os dados a cada chamada.

Essa decisão de projeto segue a filosofia de encapsulamento da programação orientada a objetos, permitindo que o objeto `LSSolver` mantenha o estado do sistema ao longo de diversas operações (eliminação, cálculo do determinante, inversa etc.). Além disso, torna mais simples a implementação de funcionalidades auxiliares como

cópia da matriz original, verificação de pivôs nulos, ou restauração da forma original após transformações.

Dessa forma, a matriz aumentada passa a ser o núcleo da instância da classe, manipulada internamente por todos os algoritmos numéricos definidos.

- **A função de exibição** `print_matrix`, ainda que não integre diretamente os algoritmos numéricos implementados, exerce um papel relevante como ferramenta de apoio ao desenvolvimento, depuração e validação visual dos resultados. Sua definição é a seguinte:

```
1 def print_matrix(self, matrix=None, format_spec="7.2"):  
2     if matrix is None:  
3         matrix = self.augMat  
4  
5     for row in matrix:  
6         for value in row:  
7             print(f"{value:{format_spec}f}", end=" ")  
8         print()
```

O objetivo dessa função é prover uma forma genérica e flexível de imprimir matrizes, sem a necessidade de estruturas ou bibliotecas adicionais. Caso nenhum argumento seja fornecido, a função imprime por padrão a matriz aumentada `self.augMat`, que é central nas operações da classe. No entanto, também é possível utilizá-la para visualizar, de maneira uniforme, outras matrizes geradas durante os procedimentos, como:

- A matriz de coeficientes A ;
- A matriz identidade I , usada na construção da matriz aumentada para obtenção da inversa;
- A matriz inversa A^{-1} , calculada via eliminação de Gauss-Jordan;
- A forma triangular superior obtida na eliminação de Gauss;
- A matriz aumentada final em forma reduzida por linhas (RREF);

A utilização do parâmetro `format_spec` permite ainda controlar o formato de exibição dos números, o que é útil tanto para propósitos estéticos quanto para análise de precisão numérica.

Portanto, embora simples, essa função oferece um recurso versátil que contribui para a transparência e compreensão do processo numérico, especialmente quando o código está sendo usado para fins didáticos ou para depuração durante a construção de algoritmos mais complexos.

- **A função Norma de Frobenius** `frobenius_norm()` implementa a seguinte definição matemática para uma matriz $A \in \mathbb{R}^{m \times n}$, dada por:

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2}$$

No código, temos:

```

1 def frobenius_norm(self, matrix=None):
2     if matrix is None:
3         matrix = self.coef_matrix()
4
5     total = 0.0
6     for row in matrix:
7         for value in row:
8             total += value ** 2
9
10    return math.sqrt(total)

```

Se nenhum argumento for fornecido, a função aplica o cálculo à matriz de coeficientes do sistema (obtida via `self.coef_matrix()`). Contudo, ela também é projetada para funcionar com qualquer matriz fornecida explicitamente como argumento, o que amplia sua utilidade.

Na implementação, o código percorre todos os elementos da matriz somando seus quadrados, e em seguida retorna a raiz quadrada dessa soma, conforme a definição teórica.

A norma de Frobenius é amplamente utilizada na análise de estabilidade de algoritmos numéricos. Em nosso contexto, ela é empregada principalmente no cálculo do **número de condição** da matriz, um indicador crítico da sensibilidade do sistema a pequenas perturbações nos dados.

Apesar de sua simplicidade, essa função é uma das peças-chave para análise da qualidade numérica das soluções obtidas.

- A função `zeros_matrix` tem por objetivo criar uma matriz de dimensões arbitrárias ($m \times n$) preenchida com o valor zero. Ela é utilizada frequentemente em etapas que exigem a alocação de espaço para uma nova matriz – como, por exemplo, na extração da matriz de coeficientes a partir de uma matriz aumentada ou na construção de matrizes auxiliares.

```
1 @staticmethod
2 def zeros_matrix(rows, cols):
3     return [[0.0 for _ in range(cols)] for _ in range(rows)]
```

Essa implementação retorna uma lista de listas contendo apenas o valor `0.0`, com precisão de ponto flutuante, o que é consistente com as operações numéricas realizadas posteriormente no código.

Uso do `@staticmethod`

Um ponto que merece destaque é a nossa escolha de declarar essa função como um método estático através do `@staticmethod`. Isso significa que a função pertence à classe `LSSolver`, mas não depende de nenhum estado interno da instância (isto é, não acessa `self` ou qualquer atributo do objeto). Ela atua apenas sobre os parâmetros fornecidos explicitamente, sendo, portanto, funcionalmente independente do objeto.

Por que utilizar `@staticmethod`?

A utilização de `@staticmethod` aqui tem como principais motivações:

- **Organização lógica:** embora a função não dependa de atributos da classe, sua utilidade está diretamente ligada ao contexto da resolução de sistemas lineares, justificando sua presença na classe `LSSolver`.
- **Evita criação de instância:** pode ser chamada diretamente pela classe, sem necessidade de criar um objeto. Exemplo:

```
1 M = LSSolver.zeros_matrix(3, 3)
```

- **Clareza semântica:** O uso do decorator explicita ao leitor e ao interpretador do código que essa função não altera e nem acessa o estado interno do objeto.

Nota sobre decorators

Decoradores em Python, como `@staticmethod`, são funções especiais que modificam o comportamento de funções ou métodos. Ao utilizarmos `@staticmethod`, estamos dizendo ao interpretador que a função não deve receber o primeiro argumento padrão `self`, o que evita o uso desnecessário de atributos internos da classe.

Resumo: Embora a função em si seja estruturalmente simples, a decisão de implementá-la como método estático reflete uma boa prática de encapsulamento e organização modular, mantendo o código coeso e reutilizável dentro do escopo conceitual da classe.

- A função `coef_matrix` tem como objetivo extrair, a partir da matriz aumentada armazenada em `self.augMat`, a matriz de coeficientes do sistema linear – isto é, todos os elementos exceto a última coluna, que representa os termos independentes.

```
1 def coef_matrix(self):
2     rows = len(self.augMat)
3     cols = len(self.augMat[0])
4     MC = self.zeros_matrix(rows, cols - 1)
5
6     for i in range(rows):
7         for j in range(cols - 1):
8             MC[i][j] = self.augMat[i][j]
9
10    return MC
```

Descrição passo a passo:

- Primeiramente, obtêm-se as dimensões da matriz aumentada, presumida como uma matriz de n linhas por $n + 1$ colunas.
- Em seguida, cria-se uma nova matriz MC , com dimensões $n \times n$, preenchida inicialmente com zeros, utilizando a função auxiliar `zeros_matrix`.
- Por fim, realiza-se a cópia dos elementos da matriz aumentada, excluindo a última coluna, que corresponde ao vetor de termos independentes.

Justificativa da Função

Esta função centraliza a lógica de separação entre a matriz de coeficientes e o vetor de constantes, uma prática importante tanto para organização quanto para reuso do código. Várias operações posteriores, como o cálculo da norma de Frobenius, do determinante, da inversa e do número de condição, dependem exclusivamente da matriz de coeficientes. Ter uma função dedicada evita repetições e inconsistências.

Aspecto de design relevante:

É importante observar o uso da função interna `self.zeros_matrix` para inicializar a nova matriz de coeficientes. Essa escolha mantém a consistência com o restante da classe, além de reforçar o princípio da *coesão interna* — isto é, o uso de métodos internos para tarefas comuns, evitando duplicação de lógica de construção de estruturas matriciais.

Resumo: A função `coef_matrix` representa uma etapa fundamental da preparação algébrica do sistema linear. Sua implementação contribui tanto para a clareza quanto para a reutilização eficiente do código ao longo dos métodos que exigem apenas a parte quadrada da matriz aumentada.

- **A função `_pivot_check_and_swap`** implementa uma etapa fundamental na aplicação segura dos métodos de Eliminação de Gauss e Gauss-Jordan: a verificação da viabilidade do pivô na linha atual da matriz.

```
1 def _pivot_check_and_swap(self, row_index):
2     n = len(self.augMat)
3
4     if self.augMat[row_index][row_index] == 0:
5         for j in range(row_index + 1, n):
6             if self.augMat[j][row_index] != 0:
7                 self.augMat[row_index], self.augMat[j] = self.augMat[j], self.
                        augMat[row_index]
8                 return # troca feita com sucesso
9     raise ValueError("Matriz singular! Nao eh possivel aplicar o metodo (
                        pivo zero).")
```

Objetivo: Garantir que o elemento da diagonal principal (o pivô) seja diferente de

zero antes da divisão associada à eliminação. Quando o pivô é nulo, uma troca de linhas é tentada com uma linha inferior que possua um pivô válido. Se nenhuma linha atender a esse critério, a matriz é considerada *singular* e a execução do método é interrompida.

Importância da Função:

- O método de Eliminação de Gauss (e sua variação Gauss-Jordan) depende criticamente da divisão pelo pivô. Um valor zero nessa posição resultaria em erro aritmético ou comportamento indefinido.
- A troca de linhas (pivotamento parcial) é uma técnica clássica para lidar com pivôs nulos ou numericamente instáveis, aumentando a robustez do método.
- A função contribui diretamente para a estabilidade do algoritmo e para a capacidade de lidar com sistemas que, à primeira vista, parecem mal colocados.

Design e Nomenclatura:

- O nome da função começa com um sublinhado (`_`), indicando que se trata de um método "privado", ou seja, voltado para uso interno da classe.
- A função modifica a matriz aumentada `self.augMat` diretamente, sendo usada como passo auxiliar dentro de métodos maiores, como `gaussian_elimination` e `gauss_jordan_elimination`.

Tratamento de Erro: Caso não seja possível encontrar uma linha com pivô válido, a função lança uma exceção do tipo `ValueError`, com a mensagem *"Matriz singular!..."*. Isso previne a continuação do algoritmo sobre um sistema que não admite solução única.

Resumo: Esta função representa uma etapa crucial para assegurar a correção numérica dos métodos de resolução de sistemas lineares. Seu uso permite evitar divisões por zero, melhora a estabilidade do algoritmo e assegura a detecção precoce de casos singulares.

• Eliminação de Gauss e Gauss-Jordan

Os métodos `gaussian_elimination()` e `gauss_jordan_elimination()` implemen-

tam, respectivamente, a transformação da matriz aumentada para a forma triangular superior (REF) e para a forma escalonada reduzida por linhas (RREF). O processo segue os passos canônicos: normalização do pivô, eliminação das variáveis abaixo (Gauss) e acima (Gauss-Jordan) da diagonal principal, e eventual troca de linhas quando o pivô é nulo (`_pivot_check_and_swap`).

• Cálculo do Determinante

O método `determinant()` calcula o determinante da matriz de coeficientes por meio da eliminação de Gauss, utilizando o produto dos elementos da diagonal principal da matriz triangularizada. A contagem de trocas de linha é usada para ajustar o sinal do determinante, conforme a propriedade:

$$\det(A) = (-1)^s \prod_{i=1}^n a_{ii}$$

onde s representa o número de trocas de linha.

• Matriz Inversa

O método `inverse()` calcula a inversa da matriz de coeficientes aplicando o método de Gauss-Jordan sobre a matriz aumentada $[A | I]$, onde I é a matriz identidade. Após o processo de eliminação, a parte direita da matriz transformada corresponde à inversa A^{-1} , assumindo que A seja quadrada e inversível.

• Número de Condição

A sensibilidade da solução de um sistema linear é medida pelo número de condição da matriz de coeficientes. O método `condition_number()` calcula:

$$\kappa_F = \|A\|_F \cdot \|A^{-1}\|_F$$

Valores elevados de κ_F (tipicamente maiores que 10^2) indicam que pequenas variações nos dados podem causar grandes perturbações na solução — um sistema numericamente instável.

• Validação do Sistema Linear

O método `validate_system()` consolida os testes essenciais para avaliar se o sistema é **bem-posto** (isto é, possui solução única) e se é **estável** numericamente. Os critérios adotados são:

1. A matriz de coeficientes deve ser quadrada ($n \times n$).

2. O determinante deve ser diferente de zero ($\det A \neq 0$).
3. O número de condição não deve ultrapassar um limiar pré-definido (por padrão, 100).

O retorno da função é um dicionário com os valores do determinante, número de condição, um aviso de estabilidade e uma mensagem interpretativa sobre a confiabilidade da solução.

A seguir exibimos o código `solver.py` na íntegra.

Código: LS_Lib/solver.py

```
1 import math
2
3 class LSSolver:
4     def __init__(self, augMat):
5         self.augMat = augMat
6
7     def print_matrix(self, matrix=None, format_spec="7.2"):
8         if matrix is None:
9             matrix = self.augMat
10
11         for row in matrix:
12             for value in row:
13                 print(f"{value:{format_spec}f}", end=" ")
14             print()
15
16     def frobenius_norm(self, matrix=None):
17         """
18         Calcula a norma de Frobenius da matriz fornecida.
19         Se nenhuma matriz for fornecida, calcula para self.augMat.
20
21         :param matrix: matriz (lista de listas) opcional
22         :return: norma de Frobenius (float)
23         """
24         if matrix is None:
25             matrix = self.coef_matrix()
26
27         total = 0.0
28         for row in matrix:
29             for value in row:
30                 total += value ** 2
31
32         return math.sqrt(total)
33
34     @staticmethod
35     def zeros_matrix(rows, cols):
36         """
37         Cria uma matriz preenchida com zeros.
```

```

38     :param rows: numero de linhas
39     :param cols: numero de colunas
40     :return: matriz de zeros (list of lists)
41     """
42     return [[0.0 for _ in range(cols)] for _ in range(rows)]
43
44 def coef_matrix(self):
45     """
46     Recupera a matriz de coeficientes (sem a coluna de termos independentes)
47     :return: Matriz de coeficientes (list of lists)
48     """
49     # Recuperando as dimensoes da matriz aumentada
50     rows = len(self.augMat)
51     cols = len(self.augMat[0])
52
53     # Criando nova matriz de zeros para os coeficientes
54     MC = self.zeros_matrix(rows, cols - 1)
55
56     # Copiando os coeficientes (tudo exceto a ultima coluna)
57     for i in range(rows):
58         for j in range(cols - 1):
59             MC[i][j] = self.augMat[i][j]
60
61     return MC
62
63 def _pivot_check_and_swap(self, row_index):
64     """
65     Verifica se o pivo na linha 'row_index' eh zero.
66     Se for, tenta trocar com uma linha abaixo que tenha pivo nao-nulo.
67     Lanca um erro se a matriz for singular.
68     """
69     n = len(self.augMat)
70
71     if self.augMat[row_index][row_index] == 0:
72         for j in range(row_index + 1, n):
73             if self.augMat[j][row_index] != 0:
74                 self.augMat[row_index], self.augMat[j] = self.augMat[j], self.
                    augMat[row_index]
75         return # troca feita com sucesso

```

```
76         raise ValueError("Matriz singular! Nao eh possivel aplicar o metodo (
77             pivo zero).")
78
79     def gaussian_elimination(self):
80         """
81         Executa o metodo de Eliminacao de Gauss para triangular a matriz aumentada.
82         Modifica self.augMat in-place.
83         """
84         n = len(self.augMat)
85         m = len(self.augMat[0])
86         num_swaps = 0
87
88         for i in range(n):
89             swapped = self._pivot_check_and_swap(i)
90             if swapped:
91                 num_swaps += 1
92
93             pivot = self.augMat[i][i]
94
95             for j in range(i + 1, n):
96                 # Atencao: divide pelo pivo apenas para calcular coeficiente correto
97                 coef = self.augMat[j][i] / pivot
98                 # O laço a seguir pode otimizar começando de i, ja que antes eh zero
99                 for k in range(i, m):
100                     self.augMat[j][k] -= coef * self.augMat[i][k]
101
102         return num_swaps
103
104     def determinant(self):
105         # Extrair matriz coeficiente
106         coef = self.coef_matrix()
107
108         # Salvar a matriz original
109         original_augMat = self.augMat
110
111         # Substituir por copia para nao modificar o original
112         self.augMat = [row[:] for row in coef]
113
114         # Garantir matriz quadrada
```

```

114     n = len(self.augMat)
115     if any(len(row) != n for row in self.augMat):
116         self.augMat = original_augMat
117         raise ValueError("Matriz de coeficientes nao eh quadrada. Determinante
118                             indefinido.")
119
120     # Executar eliminacao para triangularizacao e contar swaps
121     num_swaps = self.gaussian_elimination()
122
123     # Calcular produto da diagonal
124     det = 1
125     for i in range(n):
126         det *= self.augMat[i][i]
127
128     # Ajustar sinal pelo numero de swaps
129     if num_swaps % 2 == 1:
130         det = -det
131
132     # Copiar a matriz triangular para retornar
133     triangular = [row[:] for row in self.augMat]
134
135     # Restaurar matriz original
136     self.augMat = original_augMat
137
138     return det, triangular
139
140 def gauss_jordan_elimination(self):
141     """
142     Executa o metodo de Gauss-Jordan para transformar a matriz aumentada
143     em forma reduzida por linhas (Row Reduced Echelon Form - RREF).
144     Modifica self.augMat in-place.
145     """
146     n = len(self.augMat)
147     m = len(self.augMat[0])
148
149     for i in range(n):
150         self._pivot_check_and_swap(i)
151
152         pivot = self.augMat[i][i]

```

```
152         for k in range(m):
153             self.augMat[i][k] /= pivot
154
155         for j in range(n):
156             if i != j:
157                 coef = self.augMat[j][i]
158                 for k in range(m):
159                     self.augMat[j][k] -= coef * self.augMat[i][k]
160
161     def inverse(self):
162         """
163         Calcula a inversa da matriz de coeficientes usando Gauss-Jordan.
164         :return: Matriz inversa (list of lists)
165         :raises ValueError: se a matriz nao for inversivel
166         """
167         A = self.coef_matrix()
168         n = len(A)
169
170         # Verificar se A eh quadrada
171         if any(len(row) != n for row in A):
172             raise ValueError("A matriz de coeficientes nao eh quadrada. Inversa
173                               indefinida.")
174
175         # Verificar se o determinante eh zero
176         det, _ = self.determinant()
177         if det == 0:
178             raise ValueError("Matriz nao eh inversivel (determinante zero).")
179
180         # Criar identidade
181         I = [[float(i == j) for j in range(n)] for i in range(n)]
182
183         # Construir matriz aumentada [A | I]
184         augmented = [A[i] + I[i] for i in range(n)]
185
186         # Salvar a matriz original
187         original_augMat = self.augMat
188         self.augMat = [row[:] for row in augmented]
189
190     try:
```

```

190         self.gauss_jordan_elimination()
191     except ValueError:
192         self.augMat = original_augMat
193         raise
194
195     # Extrair a parte direita (matriz inversa)
196     inverse_mat = [row[n:] for row in self.augMat]
197
198     # Restaurar matriz original
199     self.augMat = original_augMat
200
201     return inverse_mat
202
203 def condition_number(self):
204     """
205     Calcula o numero de condicao da matriz de coeficientes com base na norma de
206     Frobenius e do valor da inversa da matriz.
207
208     :return: numero de condicao (float)
209     :raises ValueError: se a matriz nao for inversivel
210     """
211     norm_A = self.frobenius_norm() # ||A||_F
212
213     try:
214         inverse_A = self.inverse() # A-1
215     except ValueError as e:
216         raise ValueError(f"Nao eh possivel calcular o numero de condicao: {e}")
217
218     norm_inv_A = self.frobenius_norm(inverse_A) # ||A-1||_F
219
220     return norm_A * norm_inv_A
221
222 def validate_system(self, condition_threshold=1e2):
223     """
224     Verifica se o sistema linear eh bem posto e estavel numericamente.
225     Critérios:
226     - Matriz quadrada e determinante diferente de zero (sistema bem posto)
227     - Numero de condicao nao excessivamente alto (evita instabilidade)
228

```

```
229         :param condition_threshold: limite aceitavel para o numero de condicao (
                default: 100)
230
231         :return: dicionario com status e mensagens
232
233         :raises ValueError: se o sistema nao for bem posto
234
235         """
236         report = {}
237
238         # Verificar se a matriz de coeficientes eh quadrada
239         A = self.coef_matrix()
240         n = len(A)
241         if any(len(row) != n for row in A):
242             raise ValueError("Matriz de coeficientes nao eh quadrada. Sistema mal
                posto.")
243
244         # Verificar determinante
245         det, _ = self.determinant()
246         report["determinant"] = det
247
248         if det == 0:
249             raise ValueError("Determinante nulo. Sistema nao possui solucao unica (
                mal posto).")
250
251         else:
252             report["well_posed"] = True
253
254         # Calcular numero de condicao
255         kappa = self.condition_number()
256         report["condition_number"] = kappa
257
258         if kappa > condition_threshold:
259             report["sensitivity_warning"] = True
260             report["message"] = (
261                 f"Sistema eh sensivel a perturbacoes. "
262                 f"\nNumero de condicao alto (k = {kappa:.2f} > {condition_threshold
                :.0f})."
263             )
264         else:
265             report["sensitivity_warning"] = False
266             report["message"] = (
267                 f"Sistema considerado numericamente estavel. "
```

```
264         f"k = {kappa:.2f}"
265     )
266
267     return report
```

Código de execução principal: gauss_01.py

O script gauss_01.py representa a aplicação prática da classe LSSolver, funcionando como ponto de entrada para a análise e resolução de um sistema linear definido por uma matriz aumentada.

```
1 from LS_Lib.solver import LSSolver
```

A importação da classe LSSolver permite utilizar todas as funcionalidades implementadas no módulo solver.py. O restante do script segue as seguintes etapas principais:

- **Definição do sistema:** a matriz aumentada `matrix` representa o sistema linear $A|b$ a ser analisado.
- **Instanciação do resolvedor:** um objeto LSSolver é criado com a matriz aumentada como argumento.
- **Validação do sistema:** a função `validate_system()` avalia se o sistema é bem posto e numericamente estável, verificando:
 - se a matriz de coeficientes é quadrada e invertível (determinante diferente de zero);
 - se o número de condição está dentro de um limite aceitável.
- **Exibição da matriz aumentada:** a matriz original é impressa no console com o auxílio do método `print_matrix()`, oferecendo uma visualização organizada dos dados.
- **Resolução via Gauss-Jordan:** o método `gauss_jordan_elimination()` é aplicado à matriz aumentada, transformando-a em forma reduzida por linhas (RREF), que já contém a solução explícita.

- **Extração da solução:** a solução do sistema é extraída diretamente da última coluna da matriz reduzida.
- **Tratamento de exceções:** se a matriz for singular ou o sistema não for resolúvel, a execução é interrompida com uma mensagem de erro apropriada.

Esse script exemplifica uma execução completa — da análise da estabilidade à obtenção da solução — e demonstra o uso modular e robusto da classe `LSSolver`, possibilitando futuras adaptações para diferentes sistemas lineares.

Segue o código na íntegra.

Código: gauss_01.py

```
1 from LS_Lib.solver import LSSolver
2
3 if __name__ == "__main__":
4     # Matriz aumentada do sistema: A | b
5     matrix = [
6         [1, 1, 2, 9],
7         [2, 4, -3, 1],
8         [3, 6, -5, 0]
9     ]
10
11     print("Analisando o sistema linear...\n")
12
13     lss = LSSolver(matrix)
14
15     try:
16         # Etapa 1: Validar sistema
17         report = lss.validate_system()
18
19         print("Avaliacao do sistema linear proposto.")
20         print(f"Determinante: {report['determinant']:.4f}")
21         print(f"Numero de condicao (k): {report['condition_number']:.2f}")
22         print(f"Aviso de sensibilidade numerica: {report['sensitivity_warning']}")
23         print(f"Mensagem: {report['message']}\n")
24
25         # Etapa 2: Exibir a matriz aumentada original
```

```
26     print("Matriz aumentada original:")
27     lss.print_matrix()
28
29     # Etapa 3: Resolver o sistema com Gauss-Jordan
30     lss.gauss_jordan_elimination()
31
32     # Etapa 4: Exibir a matriz reduzida (RREF)
33     print("\n Matriz na forma reduzida (RREF):")
34     lss.print_matrix()
35
36     # Etapa 5: Extrair e exibir a solucao do sistema
37     solution = [row[-1] for row in lss.augMat]
38     print("\n Solucao do sistema:")
39     for i, val in enumerate(solution):
40         print(f"x{i + 1} = {val:.4f}")
41
42 except ValueError as e:
43     print(" Erro durante a resolucao do sistema:")
44     print(str(e))
```

Método de Decomposição LU

A decomposição LU constitui uma das principais técnicas da Álgebra Linear Computacional para a resolução de sistemas lineares. Sua importância decorre não apenas da possibilidade de resolver um sistema $Ax = b$, mas também da forma como ela reorganiza o processo de eliminação de Gauss em uma estrutura matricial que pode ser reutilizada em diferentes problemas [8, 9].

A ideia central pode ser compreendida a partir do próprio método de eliminação de Gauss estudado anteriormente. Ao aplicar operações elementares de eliminação a uma matriz de coeficientes A , procuramos transformá-la em uma matriz triangular superior U . Em outras palavras, a eliminação de Gauss produz uma transformação da forma

$$A \longrightarrow U.$$

Entretanto, as operações utilizadas nessa transformação não são arbitrárias: em cada etapa da eliminação são calculados *multiplicadores*, que determinam quanto de uma linha deve ser subtraído de outra para eliminar os elementos abaixo do pivô. Esses multiplicadores contêm informação sobre todo o processo de eliminação e podem ser organizados em uma matriz triangular inferior L .

Essa observação conduz à decomposição

$$A = LU,$$

quando a eliminação pode ser realizada sem a necessidade de troca de linhas. Nessa representação,

$$L = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ \ell_{21} & 1 & \cdots & 0 \\ \ell_{31} & \ell_{32} & \ddots & \vdots \\ \vdots & \vdots & \ddots & 1 \end{bmatrix}$$

é uma matriz triangular inferior com diagonal principal unitária, enquanto

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{bmatrix}$$

é uma matriz triangular superior.

A decomposição LU pode, portanto, ser entendida como uma forma de **registrar matricialmente o processo de eliminação de Gauss**: a matriz U contém o resultado da triangularização, enquanto os elementos abaixo da diagonal de L armazenam os multiplicadores empregados durante a eliminação [8, 9].

Da Eliminação de Gauss à Decomposição LU

Considere, de maneira geral, um sistema linear:

$$A\mathbf{x} = \mathbf{b},$$

com:

$$A \in \mathbb{R}^{n \times n}, \quad \mathbf{x}, \mathbf{b} \in \mathbb{R}^n.$$

Na eliminação de Gauss, o objetivo é transformar A em uma matriz triangular superior U . Para isso, na primeira etapa, utiliza-se o primeiro pivô a_{11} para eliminar os elementos abaixo dele. Para $i = 2, \dots, n$, define-se o multiplicador:

$$\ell_{i1} = \frac{a_{i1}}{a_{11}}.$$

A operação realizada sobre a linha i é então:

$$L_i \leftarrow L_i - \ell_{i1}L_1.$$

Na etapa seguinte, o mesmo procedimento é aplicado à submatriz restante, utilizando o pivô a_{22} . Para $i = 3, \dots, n$, temos:

$$\ell_{i2} = \frac{a_{i2}}{a_{22}},$$

e realizamos:

$$L_i \leftarrow L_i - \ell_{i2}L_2.$$

O processo continua até que todos os elementos abaixo da diagonal principal tenham sido anulados.

Os multiplicadores empregados em cada etapa podem ser organizados na matriz:

$$L = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ \ell_{21} & 1 & \cdots & 0 \\ \ell_{31} & \ell_{32} & \ddots & \vdots \\ \vdots & \vdots & \ddots & 1 \end{bmatrix}.$$

Dessa forma, a eliminação de Gauss deixa de ser vista apenas como uma sequência de operações sobre linhas e passa a ser interpretada como uma fatoração matricial:

$$\boxed{A = LU}.$$

Essa interpretação é particularmente importante em computação científica, pois separa duas etapas conceitualmente distintas do problema: a **fatoração da matriz** e a **resolução do sistema**. Uma vez obtidas L e U , não é necessário repetir a eliminação de Gauss para cada novo vetor $\mathbf{b}$. Em vez disso, a solução pode ser obtida por meio de dois sistemas triangulares, como será discutido posteriormente [9].

Por que a Fatoração LU é Útil?

A utilidade da decomposição LU torna-se particularmente evidente quando precisamos resolver vários sistemas que possuem a mesma matriz de coeficientes:

$$A\mathbf{x}_1 = \mathbf{b}_1, \quad A\mathbf{x}_2 = \mathbf{b}_2, \quad \dots, \quad A\mathbf{x}_m = \mathbf{b}_m.$$

Nesse caso, a fatoração de A pode ser realizada uma única vez:

$$A = LU.$$

Para cada novo vetor $\mathbf{b}_j$, a solução é obtida resolvendo os dois sistemas triangulares

$$L\mathbf{y}_j = \mathbf{b}_j$$

e

$$U\mathbf{x}_j = \mathbf{y}_j.$$

A fatoração, portanto, constitui uma etapa de *pré-processamento* que pode ser reutilizada em diferentes sistemas com a mesma matriz de coeficientes. Essa característica é uma

das razões pelas quais a decomposição LU desempenha papel central em bibliotecas de Álgebra Linear Numérica e em aplicações científicas e de engenharia [9].

Além disso, a estrutura triangular das matrizes L e U torna a resolução particularmente simples. O sistema associado a L pode ser resolvido por **substituição progressiva**, enquanto o sistema associado a U pode ser resolvido por **substituição regressiva**.

Temos, portanto, a sequência

$$Ax = b \implies LUx = b,$$

e, introduzindo um vetor auxiliar y ,

$$\begin{array}{l} Ly = b, \\ Ux = y. \end{array}$$

A decomposição LU transforma, assim, um problema de resolução de um sistema linear geral em dois problemas envolvendo matrizes triangulares.

Quando $A = LU$ não é suficiente?

A fatoração $A = LU$ apresentada até aqui pressupõe que os pivôs utilizados durante a eliminação sejam adequados. Em particular, o processo não pode prosseguir se um pivô for exatamente zero. Além disso, mesmo quando o pivô é diferente de zero, sua utilização pode produzir multiplicadores muito grandes, o que pode amplificar os efeitos do arredondamento em uma implementação computacional.

Esse problema não é apenas uma questão algébrica. Em aritmética de ponto flutuante, os números armazenados pelo computador são aproximações dos valores matemáticos. Consequentemente, operações envolvendo divisões por pivôs muito pequenos podem comprometer a qualidade numérica do resultado. O estudo da eliminação de Gauss em aritmética de ponto flutuante mostra, portanto, que a escolha dos pivôs possui papel fundamental na estabilidade do algoritmo [9, 10].

E é nesse contexto que surge o **pivotamento parcial**.

Em vez de utilizar automaticamente o elemento a_{kk} como pivô na etapa k , procuramos, na coluna k , entre as linhas ainda não eliminadas, um elemento de maior valor absoluto.

A linha correspondente é então permutada para a posição do pivô.

De forma esquemática, na etapa k , procuramos

$$p = \arg \max_{i=k, \dots, n} |a_{ik}|.$$

Se $p \neq k$, as linhas k e p são trocadas antes da continuação da eliminação.

Essa estratégia é denominada **pivotamento parcial**. Ela reduz a possibilidade de utilização de pivôs pequenos e, no contexto da eliminação de Gauss, é a estratégia padrão para melhorar a estabilidade numérica do processo [9, 10].

A Matriz de Permutação

As trocas de linhas realizadas durante o pivotamento podem ser representadas por uma matriz de permutação P . Uma matriz de permutação é obtida a partir da matriz identidade por meio de permutações de suas linhas e satisfaz:

$$P^{-1} = P^T.$$

Quando ocorrem trocas de linhas durante a eliminação, a fatoração passa a ser representada por:

$$PA = LU.$$

A presença de P é conceitualmente importante: ela registra as permutações realizadas antes ou durante a eliminação e permite distinguir claramente a matriz original A da matriz efetivamente triangularizada pelo algoritmo.

Assim, a decomposição utilizada neste material será a forma $PA = LU$, na qual:

- P registra as permutações de linhas realizadas pelo pivotamento;
- L armazena os multiplicadores da eliminação, possuindo diagonal principal unitária;
- U é a matriz triangular superior resultante da eliminação.

Essa forma da decomposição é especialmente apropriada para uma implementação computacional, pois o algoritmo pode registrar as escolhas de pivô em um vetor de índices e, posteriormente, reconstruir a permutação correspondente.

Da Fatoração à Resolução do Sistema

A presença da matriz de permutação modifica ligeiramente a resolução do sistema original:

$$A\mathbf{x} = \mathbf{b}.$$

Como:

$$PA = LU,$$

multiplicamos o sistema pela matriz P :

$$PA\mathbf{x} = P\mathbf{b}.$$

Consequentemente,

$$LU\mathbf{x} = P\mathbf{b}.$$

Introduzindo novamente um vetor auxiliar $\mathbf{y}$, podemos escrever:

$$\begin{array}{l} L\mathbf{y} = P\mathbf{b}, \\ U\mathbf{x} = \mathbf{y}. \end{array}$$

Portanto, a resolução completa de um sistema utilizando LU com pivotamento parcial pode ser visualizada como o fluxo:

$$A\mathbf{x} = \mathbf{b} \longrightarrow PA = LU \longrightarrow \begin{cases} L\mathbf{y} = P\mathbf{b}, \\ U\mathbf{x} = \mathbf{y}. \end{cases}$$

Essa sequência resume a ideia fundamental do método: primeiro fatoramos a matriz de coeficientes, levando em consideração as permutações necessárias; depois utilizamos a estrutura triangular resultante para obter a solução.

A partir dessa perspectiva, o pivotamento deixa de ser uma operação aparentemente isolada e passa a ser compreendido como parte integrante do processo de fatoração. Essa interpretação será utilizada na implementação computacional apresentada nas próximas subseções.

Decomposição LU com Pivotamento: Passo a Passo Detalhado

Nesta seção, aplicaremos o procedimento de decomposição LU com pivotamento parcial a um exemplo concreto. O objetivo não é apenas obter as matrizes P , L e U , mas compreender como cada uma delas surge durante o processo de eliminação.

Consideremos a matriz:

$$A = \begin{bmatrix} 2 & 3 & 1 \\ 4 & 7 & 7 \\ 6 & 18 & 22 \end{bmatrix}.$$

Nosso objetivo é determinar as matrizes P , L e U que satisfaçam:

$$PA = LU,$$

utilizando pivotamento parcial.

Ao longo do procedimento, utilizaremos a seguinte convenção: na etapa k , procuramos o maior elemento em módulo na coluna k , considerando apenas as linhas $k, k + 1, \dots, n$. A linha correspondente é posicionada na posição k , tornando o elemento escolhido o pivô da etapa.

Etapa 1: seleção e posicionamento do primeiro pivô

Inicialmente, consideramos a primeira coluna de A :

$$\begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}.$$

Os valores absolutos dos elementos são $|2| = 2$, $|4| = 4$ e $|6| = 6$. Portanto,

$$\max\{|2|, |4|, |6|\} = 6.$$

O maior valor absoluto está na terceira linha. Assim, o elemento:

$$a_{31} = 6$$

é escolhido como primeiro pivô.

Para posicioná-lo na primeira posição da diagonal principal, permutamos as linhas 1 e 3:

$$\begin{bmatrix} 2 & 3 & 1 \\ 4 & 7 & 7 \\ 6 & 18 & 22 \end{bmatrix} \xrightarrow{L_1 \leftrightarrow L_3} \begin{bmatrix} 6 & 18 & 22 \\ 4 & 7 & 7 \\ 2 & 3 & 1 \end{bmatrix}.$$

Denotaremos a matriz resultante por:

$$A^{(1)} = \begin{bmatrix} 6 & 18 & 22 \\ 4 & 7 & 7 \\ 2 & 3 & 1 \end{bmatrix}.$$

A primeira troca de linhas é registrada pelo algoritmo de pivotamento. Em uma implementação em Python que utiliza indexação iniciando em zero, essa troca pode ser registrada como:

$$\text{ipvt}[0] = 2.$$

É importante observar que o índice 2 corresponde à terceira linha na notação matemática usual. A distinção entre índices matemáticos $1, 2, \dots, n$ e índices computacionais $0, 1, \dots, n-1$ será mantida explicitamente para evitar ambiguidades na implementação.

Etapa 2: cálculo dos multiplicadores e eliminação

Com o pivô:

$$u_{11} = 6,$$

devemos eliminar os elementos abaixo dele.

Para a segunda linha, o multiplicador é:

$$\ell_{21} = \frac{4}{6} = \frac{2}{3}.$$

Para a terceira linha,

$$\ell_{31} = \frac{2}{6} = \frac{1}{3}.$$

Esses multiplicadores serão posteriormente armazenados na matriz L .

A operação de eliminação sobre a segunda linha é:

$$L_2 \leftarrow L_2 - \frac{2}{3}L_1.$$

Assim,

$$\begin{aligned} L_2 &= \begin{bmatrix} 4 & 7 & 7 \end{bmatrix} - \frac{2}{3} \begin{bmatrix} 6 & 18 & 22 \end{bmatrix} \\ &= \begin{bmatrix} 4 - 4 & 7 - 12 & 7 - \frac{44}{3} \end{bmatrix} \\ &= \begin{bmatrix} 0 & -5 & -\frac{23}{3} \end{bmatrix}. \end{aligned}$$

Para a terceira linha:

$$L_3 \leftarrow L_3 - \frac{1}{3}L_1,$$

resultando em

$$\begin{aligned} L_3 &= \begin{bmatrix} 2 & 3 & 1 \end{bmatrix} - \frac{1}{3} \begin{bmatrix} 6 & 18 & 22 \end{bmatrix} \\ &= \begin{bmatrix} 2 - 2 & 3 - 6 & 1 - \frac{22}{3} \end{bmatrix} \\ &= \begin{bmatrix} 0 & -3 & -\frac{19}{3} \end{bmatrix}. \end{aligned}$$

Portanto, após a primeira etapa de eliminação, temos:

$$A^{(2)} = \begin{bmatrix} 6 & 18 & 22 \\ 0 & -5 & -\frac{23}{3} \\ 0 & -3 & -\frac{19}{3} \end{bmatrix}.$$

Observe que, para a construção da fatoração LU, os multiplicadores:

$$\frac{2}{3} \quad \text{e} \quad \frac{1}{3}$$

não são descartados. Eles constituem parte da informação necessária para construir a matriz L .

Etapa 3: seleção do segundo pivô

Na segunda etapa, a primeira linha e a primeira coluna já foram tratadas. Devemos, portanto, considerar apenas a submatriz correspondente às linhas 2 e 3 e à segunda coluna:

$$\begin{bmatrix} -5 \\ -3 \end{bmatrix}.$$

Os valores absolutos são $|-5| = 5$ e $|-3| = 3$. Como $5 > 3$ o elemento:

$$a_{22} = -5$$

já é o maior elemento em módulo entre os candidatos. Portanto, não é necessária uma nova troca de linhas.

Em uma implementação com indexação iniciando em zero, registramos essa situação como:

$$\text{ipvt}[1] = 1.$$

Essa informação significa que, na segunda etapa, a linha correspondente ao índice 1 permanece na posição em que já se encontra.

Etapas 4: segundo multiplicador e eliminação

$$\ell_{32} = \frac{-3}{-5} = \frac{3}{5}.$$

Aplicamos então a operação:

$$L_3 \leftarrow L_3 - \frac{3}{5}L_2.$$

O único elemento que ainda precisa ser atualizado nessa etapa é o elemento da terceira coluna:

$$\begin{aligned} a_{33} &= -\frac{19}{3} - \frac{3}{5} \left(-\frac{23}{3} \right) \\ &= -\frac{19}{3} + \frac{23}{5}. \end{aligned}$$

Colocando os termos sob o mesmo denominador,

$$a_{33} = -\frac{95}{15} + \frac{69}{15} = -\frac{26}{15}.$$

Assim, após a segunda etapa de eliminação,

$$A^{(3)} = \begin{bmatrix} 6 & 18 & 22 \\ 0 & -5 & -\frac{23}{3} \\ 0 & 0 & -\frac{26}{15} \end{bmatrix}.$$

Neste ponto, a matriz já está na forma triangular superior. Portanto,

$$U = \begin{bmatrix} 6 & 18 & 22 \\ 0 & -5 & -\frac{23}{3} \\ 0 & 0 & -\frac{26}{15} \end{bmatrix}.$$

Etapa 5: último pivô

Na terceira etapa, resta apenas o elemento:

$$u_{33} = -\frac{26}{15}.$$

Como não existem outras linhas abaixo dele, não há possibilidade de troca. Além disso,

$$-\frac{26}{15} \neq 0,$$

portanto a fatoração pode ser concluída.

Em uma implementação com indexação iniciando em zero, podemos registrar:

$$\text{ipvt}[2] = 2.$$

Consequentemente, o vetor de índices de pivotamento associado às etapas realizadas é:

$$\text{ipvt} = [2, 1, 2].$$

É importante destacar que esse vetor representa as **trocas realizadas em cada etapa**, e não simplesmente uma nova ordenação definitiva das linhas. No exemplo considerado, a única troca efetiva ocorreu na primeira etapa:

$$L_1 \leftrightarrow L_3.$$

As etapas seguintes não produziram novas permutações.

Construção da matriz de permutação P

A troca realizada na primeira etapa pode ser representada pela matriz de permutação:

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

De fato, ao multiplicarmos A por P , as linhas 1 e 3 de A são trocadas:

$$PA = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 2 & 3 & 1 \\ 4 & 7 & 7 \\ 6 & 18 & 22 \end{bmatrix} \\ = \begin{bmatrix} 6 & 18 & 22 \\ 4 & 7 & 7 \\ 2 & 3 & 1 \end{bmatrix}.$$

Assim,

$$PA = \begin{bmatrix} 6 & 18 & 22 \\ 4 & 7 & 7 \\ 2 & 3 & 1 \end{bmatrix}.$$

A matriz P registra, portanto, a permutação necessária para que a eliminação de Gauss seja realizada com pivotamento parcial.

Construção da matriz triangular inferior L

Os multiplicadores calculados durante o processo foram:

$$\ell_{21} = \frac{2}{3}, \quad \ell_{31} = \frac{1}{3}, \quad \ell_{32} = \frac{3}{5}.$$

Como a matriz L possui diagonal principal unitária, obtemos:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{2}{3} & 1 & 0 \\ \frac{1}{3} & \frac{3}{5} & 1 \end{bmatrix}.$$

É importante observar que os elementos abaixo da diagonal principal de L correspondem exatamente aos multiplicadores utilizados durante a eliminação. Essa é uma das ideias fundamentais da decomposição LU:

$$\text{multiplicadores da eliminação} \longrightarrow \text{elementos abaixo da diagonal de } L.$$

Por sua vez, a matriz U contém a parte triangular superior obtida ao final da eliminação:

$$U = \begin{bmatrix} 6 & 18 & 22 \\ 0 & -5 & -\frac{23}{3} \\ 0 & 0 & -\frac{26}{15} \end{bmatrix}.$$

Verificação da fatoração

A fatoração obtida deve satisfazer:

$$PA = LU.$$

Podemos verificar essa igualdade calculando o produto LU :

$$\begin{aligned} LU &= \begin{bmatrix} 1 & 0 & 0 \\ \frac{2}{3} & 1 & 0 \\ \frac{1}{3} & \frac{3}{5} & 1 \end{bmatrix} \begin{bmatrix} 6 & 18 & 22 \\ 0 & -5 & -\frac{23}{3} \\ 0 & 0 & -\frac{26}{15} \end{bmatrix} \\ &= \begin{bmatrix} 6 & 18 & 22 \\ 4 & 7 & 7 \\ 2 & 3 & 1 \end{bmatrix}. \end{aligned}$$

Como:

$$PA = \begin{bmatrix} 6 & 18 & 22 \\ 4 & 7 & 7 \\ 2 & 3 & 1 \end{bmatrix},$$

concluimos que:

$$PA = LU.$$

A decomposição está, portanto, corretamente determinada.

Interpretação da fatoração

O exemplo permite visualizar as três componentes da decomposição:

$$PA = LU$$

com:

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{2}{3} & 1 & 0 \\ \frac{1}{3} & \frac{3}{5} & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 6 & 18 & 22 \\ 0 & -5 & -\frac{23}{3} \\ 0 & 0 & -\frac{26}{15} \end{bmatrix}.$$

Cada matriz possui uma função específica:

- P registra as **permutações de linhas** realizadas pelo pivotamento parcial;
- L registra os **multiplicadores da eliminação**, abaixo da diagonal principal;
- U contém a **matriz triangular superior** resultante da eliminação.

A decomposição LU com pivotamento parcial não deve, portanto, ser vista apenas como uma identidade matricial. Ela constitui uma forma compacta de registrar todo o processo de eliminação de Gauss:

$$\boxed{\text{pivotamento} + \text{multiplicadores} + \text{eliminação} \implies PA = LU.}$$

Essa interpretação será particularmente importante na implementação computacional, pois o algoritmo não precisa construir explicitamente todas as matrizes intermediárias. Os multiplicadores e os elementos de U podem ser armazenados na própria estrutura de dados utilizada para representar a matriz, enquanto as permutações podem ser registradas por meio de um vetor de índices como `ipvt`.

Resolução de um Sistema Linear pelo Método de Decomposição LU

Vamos agora resolver o sistema linear apresentado anteriormente na Equação (2.1), utilizando a **decomposição LU com pivotamento parcial**. Esse procedimento é particularmente importante do ponto de vista computacional, pois permite decompor a matriz de coeficientes em matrizes triangulares e, posteriormente, resolver dois sistemas lineares mais simples.

O sistema considerado é:

$$\begin{cases} x + y + 2z = 9 \\ 2x + 4y - 3z = 1 \\ 3x + 6y - 5z = 0 \end{cases} \quad (2.10)$$

Escrevendo o sistema na forma matricial, temos:

$$A\mathbf{x} = \mathbf{b},$$

onde

$$A = \begin{bmatrix} 1 & 1 & 2 \\ 2 & 4 & -3 \\ 3 & 6 & -5 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 9 \\ 1 \\ 0 \end{bmatrix}.$$

Na decomposição LU com pivotamento parcial, procuramos uma matriz de permutação P e duas matrizes triangulares L e U , de modo que:

$$PA = LU.$$

O pivotamento parcial consiste, em cada etapa, em selecionar como pivô o elemento de maior valor absoluto na coluna atualmente considerada. Essa estratégia evita, sempre que possível, divisões por valores muito pequenos e contribui para a estabilidade numérica da fatoração.

Passo 1: seleção do primeiro pivô

Na primeira etapa, consideramos a primeira coluna da matriz A :

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Os valores absolutos dos elementos são $|1| = 1$, $|2| = 2$ e $|3| = 3$. Portanto, o maior valor absoluto é 3, localizado na terceira linha. Assim, o primeiro pivô escolhido será:

$$a_{31} = 3.$$

Para posicionar esse elemento na primeira posição da diagonal principal, trocamos a primeira linha com a terceira linha:

$$A^{(1)} = \begin{bmatrix} 3 & 6 & -5 \\ 2 & 4 & -3 \\ 1 & 1 & 2 \end{bmatrix}.$$

Essa primeira troca de linhas é registrada na matriz de permutação P .

Passo 2: cálculo dos multiplicadores

Com o pivô $a_{11} = 3$, calculamos os multiplicadores das linhas abaixo do pivô:

$$l_{21} = \frac{a_{21}}{a_{11}} = \frac{2}{3},$$

e

$$l_{31} = \frac{a_{31}}{a_{11}} = \frac{1}{3}.$$

Esses multiplicadores são utilizados para eliminar os elementos abaixo do primeiro pivô.

Para a segunda linha, fazemos:

$$L_2 \leftarrow L_2 - \frac{2}{3}L_1.$$

Assim,

$$\begin{aligned} L_2 &= \begin{bmatrix} 2 & 4 & -3 \end{bmatrix} - \frac{2}{3} \begin{bmatrix} 3 & 6 & -5 \end{bmatrix} \\ &= \begin{bmatrix} 2 - 2 & 4 - 4 & -3 + \frac{10}{3} \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & \frac{1}{3} \end{bmatrix}. \end{aligned}$$

Para a terceira linha:

$$L_3 \leftarrow L_3 - \frac{1}{3}L_1,$$

resultando em:

$$\begin{aligned} L_3 &= \begin{bmatrix} 1 & 1 & 2 \end{bmatrix} - \frac{1}{3} \begin{bmatrix} 3 & 6 & -5 \end{bmatrix} \\ &= \begin{bmatrix} 0 & -1 & \frac{11}{3} \end{bmatrix}. \end{aligned}$$

Portanto, após a primeira etapa da eliminação, temos:

$$\begin{bmatrix} 3 & 6 & -5 \\ 0 & 0 & \frac{1}{3} \\ 0 & -1 & \frac{11}{3} \end{bmatrix}.$$

Passo 3: seleção do pivô na segunda coluna

Na segunda etapa, devemos procurar o maior valor absoluto na segunda coluna da submatriz formada pelas linhas 2 e 3:

$$\begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

Temos $|0| = 0$ e $|-1| = 1$. Logo, o maior valor absoluto é 1, localizado na terceira linha da matriz atual. Devemos, portanto, trocar as linhas 2 e 3:

$$\begin{bmatrix} 3 & 6 & -5 \\ 0 & -1 & \frac{11}{3} \\ 0 & 0 & \frac{1}{3} \end{bmatrix}.$$

Nesse ponto, o segundo pivô é:

$$u_{22} = -1.$$

Como o elemento abaixo desse pivô já é zero, não há necessidade de realizar uma nova eliminação nessa coluna. Consequentemente, o multiplicador correspondente é:

$$l_{32} = \frac{0}{-1} = 0.$$

O terceiro pivô é simplesmente:

$$u_{33} = \frac{1}{3}.$$

Portanto, a matriz triangular superior obtida ao final da fatoração é:

$$U = \begin{bmatrix} 3 & 6 & -5 \\ 0 & -1 & \frac{11}{3} \\ 0 & 0 & \frac{1}{3} \end{bmatrix}.$$

Passo 4: construção da matriz de permutação

Durante a fatora  o ocorreram duas trocas de linhas:

1. na primeira etapa, a linha 1 foi trocada com a linha 3;
2. na segunda etapa, a linha 2 foi trocada com a linha 3.

Consequentemente, a ordem final das linhas de A  :

$$(L_3, L_1, L_2).$$

Assim, a matriz de permuta  o correspondente  :

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

Aplicando essa matriz   matriz original, obtemos:

$$PA = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 & 2 \\ 2 & 4 & -3 \\ 3 & 6 & -5 \end{bmatrix},$$

ou seja,

$$PA = \begin{bmatrix} 3 & 6 & -5 \\ 1 & 1 & 2 \\ 2 & 4 & -3 \end{bmatrix}.$$

Passo 5: constru  o da matriz triangular inferior L

Os multiplicadores calculados durante a elimina  o s o armazenados na matriz triangular inferior L . Como a matriz L possui diagonal principal unit ria, temos:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix}.$$

Entretanto, é importante observar que houve uma troca de linhas entre a primeira e a segunda etapas da fatoração. Os multiplicadores já calculados devem acompanhar as linhas correspondentes. Assim, após o pivotamento, temos:

$$l_{21} = \frac{1}{3}, \quad l_{31} = \frac{2}{3}, \quad l_{32} = 0.$$

Portanto,

$$L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{3} & 1 & 0 \\ \frac{2}{3} & 0 & 1 \end{bmatrix}.$$

Podemos verificar a fatoração calculando LU :

$$LU = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{3} & 1 & 0 \\ \frac{2}{3} & 0 & 1 \end{bmatrix} \begin{bmatrix} 3 & 6 & -5 \\ 0 & -1 & \frac{11}{3} \\ 0 & 0 & \frac{1}{3} \end{bmatrix}.$$

Efetuando o produto matricial, encontramos:

$$LU = \begin{bmatrix} 3 & 6 & -5 \\ 1 & 1 & 2 \\ 2 & 4 & -3 \end{bmatrix}.$$

Como:

$$PA = \begin{bmatrix} 3 & 6 & -5 \\ 1 & 1 & 2 \\ 2 & 4 & -3 \end{bmatrix},$$

temos finalmente:

$$PA = LU.$$

A fatoração LU está, portanto, confirmada.

Passo 6: transformação do sistema original

Temos originalmente:

$$Ax = b.$$

Como:

$$PA = LU,$$

podemos multiplicar o sistema pela matriz de permutação P :

$$PA\mathbf{x} = P\mathbf{b}.$$

Como $PA = LU$, segue que:

$$LU\mathbf{x} = P\mathbf{b}.$$

Para simplificar a resolução, introduzimos um vetor auxiliar $\mathbf{y}$, definido por:

$$U\mathbf{x} = \mathbf{y}.$$

Dessa forma, o problema original é dividido em dois sistemas triangulares:

$$\boxed{L\mathbf{y} = P\mathbf{b}}$$

e

$$\boxed{U\mathbf{x} = \mathbf{y}.$$

Essa é uma das principais vantagens computacionais da decomposição LU: uma vez realizada a fatoração, a resolução do sistema pode ser reduzida a duas substituições, uma progressiva e outra regressiva.

Passo 7: cálculo de $P\mathbf{b}$

Temos:

$$\mathbf{b} = \begin{bmatrix} 9 \\ 1 \\ 0 \end{bmatrix}.$$

Aplicando a matriz de permutação:

$$P\mathbf{b} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 9 \\ 1 \\ 0 \end{bmatrix}.$$

Portanto,

$$P\mathbf{b} = \begin{bmatrix} 0 \\ 9 \\ 1 \end{bmatrix}.$$

Assim, devemos resolver inicialmente:

$$L\mathbf{y} = P\mathbf{b},$$

isto é,

$$\begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{3} & 1 & 0 \\ \frac{2}{3} & 0 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 9 \\ 1 \end{bmatrix}.$$

Passo 8: substituição progressiva

Como L é uma matriz triangular inferior, podemos determinar os elementos de $\mathbf{y}$ de cima para baixo.

A primeira equação é:

$$y_1 = 0.$$

A segunda equação é:

$$\frac{1}{3}y_1 + y_2 = 9.$$

Como $y_1 = 0$, temos:

$$y_2 = 9.$$

Finalmente, a terceira equação é:

$$\frac{2}{3}y_1 + y_3 = 1.$$

Como $y_1 = 0$, obtemos:

$$y_3 = 1.$$

Portanto,

$$\mathbf{y} = \begin{bmatrix} 0 \\ 9 \\ 1 \end{bmatrix}.$$

Passo 9: substituição regressiva

Agora devemos resolver:

$$U\mathbf{x} = \mathbf{y},$$

ou seja,

$$\begin{bmatrix} 3 & 6 & -5 \\ 0 & -1 & \frac{11}{3} \\ 0 & 0 & \frac{1}{3} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 0 \\ 9 \\ 1 \end{bmatrix}.$$

Como U é triangular superior, começamos pela última equação.

A terceira equação é:

$$\frac{1}{3}z = 1.$$

Multiplicando ambos os lados por 3, encontramos:

$$\boxed{z = 3}.$$

A segunda equação é:

$$-y + \frac{11}{3}z = 9.$$

Substituindo $z = 3$:

$$-y + \frac{11}{3}(3) = 9,$$

portanto,

$$-y + 11 = 9,$$

e conseqüentemente,

$$\boxed{y = 2}.$$

Finalmente, utilizamos a primeira equação:

$$3x + 6y - 5z = 0.$$

Substituindo $y = 2$ e $z = 3$:

$$3x + 6(2) - 5(3) = 0.$$

Logo,

$$3x + 12 - 15 = 0,$$

de onde:

$$3x - 3 = 0.$$

Assim,

$$x = 1.$$

Portanto, a solução obtida pela decomposição LU com pivotamento parcial é:

$$x = 1, \quad y = 2, \quad z = 3.$$

Resumo computacional da decomposição

O procedimento realizado pode ser resumido pela seguinte sequência:

$$A \xrightarrow{\text{pivotamento parcial}} PA \xrightarrow{\text{eliminação}} LU,$$

com:

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{3} & 1 & 0 \\ \frac{2}{3} & 0 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 3 & 6 & -5 \\ 0 & -1 & \frac{11}{3} \\ 0 & 0 & \frac{1}{3} \end{bmatrix}.$$

A resolução do sistema ocorre então em duas etapas:

$$Ly = Pb$$

e

$$Ux = y.$$

Neste exemplo,

$$Pb = \begin{bmatrix} 0 \\ 9 \\ 1 \end{bmatrix}, \quad y = \begin{bmatrix} 0 \\ 9 \\ 1 \end{bmatrix},$$

e, finalmente,

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Assim, a decomposição LU transforma a resolução do sistema original em duas operações elementares sobre sistemas triangulares: uma **substituição progressiva** para determinar y e uma **substituição regressiva** para determinar x .



Exercícios: Aplicação da Decomposição LU

Os exercícios a seguir têm como objetivo consolidar a aplicação da decomposição LU na resolução de sistemas lineares. Em particular, devem ser observadas as etapas de fatoração, o papel do pivotamento parcial, a resolução dos sistemas triangulares e a verificação dos resultados.

Sempre que necessário, considere a forma:

$$PA = LU,$$

onde P é uma matriz de permutação, L é uma matriz triangular inferior com diagonal principal unitária e U é uma matriz triangular superior.

Para a resolução de um sistema:

$$Ax = b,$$

utilize a sequência:

$$PAx = Pb \implies LUx = Pb,$$

introduzindo o vetor auxiliar y :

$$\begin{array}{l} Ly = Pb, \\ Ux = y. \end{array}$$

Quando houver pivotamento, registre as trocas de linhas e determine a matriz P correspondente.

 Lista de Exercícios

2.7 Considere o sistema linear:

$$\begin{cases} 4x + y = 9 \\ 2x + 3y = 7 \end{cases}$$

Resolva o sistema utilizando a decomposição LU. Determine explicitamente as matrizes L e U , resolva os dois sistemas triangulares e verifique a solução obtida no sistema original.

Gabarito:

Escrevemos o sistema na forma matricial:

$$A\mathbf{x} = \mathbf{b},$$

com:

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 9 \\ 7 \end{bmatrix}.$$

Como o primeiro elemento da primeira coluna é adequado como pivô,

$$a_{11} = 4,$$

não é necessário realizar pivotamento.

O multiplicador utilizado para eliminar o elemento abaixo do primeiro pivô é

$$\ell_{21} = \frac{2}{4} = \frac{1}{2}.$$

Assim,

$$L_2 \leftarrow L_2 - \frac{1}{2}L_1.$$

Aplicando a operação,

$$\begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} \longrightarrow \begin{bmatrix} 4 & 1 \\ 0 & \frac{5}{2} \end{bmatrix}.$$

Portanto,

$$U = \begin{bmatrix} 4 & 1 \\ 0 & \frac{5}{2} \end{bmatrix}.$$

O multiplicador utilizado na eliminação constitui o elemento abaixo da diagonal principal de L . Como a diagonal de L é unitária:

$$L = \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix}.$$

Podemos verificar diretamente que:

$$LU = \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} 4 & 1 \\ 0 & \frac{5}{2} \end{bmatrix} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} = A.$$

Portanto,

$$A = LU.$$

Como não houve permutação,

$$P = I.$$

Agora resolvemos

$$Ly = \mathbf{b}.$$

Temos:

$$\begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 9 \\ 7 \end{bmatrix}.$$

A primeira equação fornece:

$$y_1 = 9.$$

A segunda equação é:

$$\frac{1}{2}y_1 + y_2 = 7.$$

Substituindo $y_1 = 9$,

$$\frac{9}{2} + y_2 = 7,$$

de modo que:

$$y_2 = \frac{5}{2}.$$

Assim,

$$\mathbf{y} = \begin{bmatrix} 9 \\ \frac{5}{2} \end{bmatrix}.$$

Agora resolvemos:

$$U\mathbf{x} = \mathbf{y},$$

isto é,

$$\begin{bmatrix} 4 & 1 \\ 0 & \frac{5}{2} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 9 \\ \frac{5}{2} \end{bmatrix}.$$

Da segunda equação,

$$\frac{5}{2}y = \frac{5}{2},$$

portanto,

$$y = 1.$$

Substituindo na primeira equação,

$$4x + 1 = 9,$$

de onde:

$$x = 2.$$

Logo,

$$\boxed{x = 2, \quad y = 1}.$$

Verificação:

$$4(2) + 1 = 9,$$

e

$$2(2) + 3(1) = 7.$$

As duas equações são satisfeitas.



2.8 Considere o sistema:

$$\begin{cases} y + 2z = 5 \\ x + y + 2z = 5 \\ 2x + 3y + 4z = 9 \end{cases}$$

Resolva o sistema utilizando a decomposição LU com **pivotamento parcial**.

Determine:

- (a) as trocas de linhas realizadas;
- (b) a matriz de permutação P ;

- (c) as matrizes L e U ;
- (d) a solução do sistema;
- (e) a verificação de $PA = LU$.

Registre, ainda, o vetor de pivoteamento utilizando indexação *zero-based*, como em uma implementação computacional.

Gabarito:

A forma matricial do sistema é:

$$Ax = b,$$

com:

$$A = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 1 & 2 \\ 2 & 3 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ 5 \\ 9 \end{bmatrix}.$$

1. Primeira etapa do pivotamento

Na primeira coluna temos:

$$\begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}.$$

O maior valor absoluto é:

$$|2| = 2,$$

localizado na terceira linha. Portanto, a primeira troca é:

$$L_1 \leftrightarrow L_3.$$

Obtemos:

$$\begin{bmatrix} 2 & 3 & 4 \\ 1 & 1 & 2 \\ 0 & 1 & 2 \end{bmatrix}.$$

Em indexação *zero-based*,

$$\text{ipvt}[0] = 2.$$

O primeiro pivô é:

$$u_{11} = 2.$$

Os multiplicadores são:

$$\ell_{21} = \frac{1}{2}, \quad \ell_{31} = 0.$$

Aplicamos:

$$L_2 \longleftarrow L_2 - \frac{1}{2}L_1.$$

Assim,

$$\begin{bmatrix} 2 & 3 & 4 \\ 1 & 1 & 2 \\ 0 & 1 & 2 \end{bmatrix} \longrightarrow \begin{bmatrix} 2 & 3 & 4 \\ 0 & -\frac{1}{2} & 0 \\ 0 & 1 & 2 \end{bmatrix}.$$

2. Segunda etapa do pivotamento

Consideramos agora a segunda coluna da submatriz restante:

$$\begin{bmatrix} -\frac{1}{2} \\ 1 \end{bmatrix}.$$

Como:

$$|1| > \left| -\frac{1}{2} \right|,$$

devemos trocar as linhas 2 e 3:

$$L_2 \leftrightarrow L_3.$$

Obtemos:

$$\begin{bmatrix} 2 & 3 & 4 \\ 0 & 1 & 2 \\ 0 & -\frac{1}{2} & 0 \end{bmatrix}.$$

Em indexação *zero-based*,

$$\text{ipvt}[1] = 2.$$

O segundo pivô é:

$$u_{22} = 1.$$

O multiplicador da terceira linha é:

$$\ell_{32} = \frac{-\frac{1}{2}}{1} = -\frac{1}{2}.$$

Portanto,

$$L_3 \longleftarrow L_3 + \frac{1}{2}L_2.$$

Logo,

$$\begin{bmatrix} 2 & 3 & 4 \\ 0 & 1 & 2 \\ 0 & -\frac{1}{2} & 0 \end{bmatrix} \longrightarrow \begin{bmatrix} 2 & 3 & 4 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}.$$

Assim,

$$U = \begin{bmatrix} 2 & 3 & 4 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}.$$

3. Matriz de permutação

As trocas realizadas foram:

$$L_1 \leftrightarrow L_3$$

e

$$L_2 \leftrightarrow L_3.$$

A permutação final coloca as linhas originais na ordem:

$$(L_3, L_1, L_2).$$

Portanto,

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

O vetor de pivoteamento, utilizando a convenção adotada no exercício, é:

$$\text{ipvt} = [2, 2, 2].$$

Observe que esse vetor registra a linha escolhida em cada etapa, e não simplesmente a permutação final.

4. Construção de L

Os multiplicadores efetivamente associados à fatoração são:

$$\ell_{21} = 0, \quad \ell_{31} = \frac{1}{2}, \quad \ell_{32} = -\frac{1}{2}.$$

Logo,

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \frac{1}{2} & -\frac{1}{2} & 1 \end{bmatrix}.$$

Podemos verificar:

$$\begin{aligned} LU &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \frac{1}{2} & -\frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} 2 & 3 & 4 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 2 & 3 & 4 \\ 0 & 1 & 2 \\ 1 & 1 & 2 \end{bmatrix}. \end{aligned}$$

Por outro lado,

$$PA = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 2 \\ 1 & 1 & 2 \\ 2 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 4 \\ 0 & 1 & 2 \\ 1 & 1 & 2 \end{bmatrix}.$$

Portanto,

$$PA = LU.$$

5. Resolução do sistema

Primeiro calculamos:

$$P\mathbf{b} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 5 \\ 5 \\ 9 \end{bmatrix} = \begin{bmatrix} 9 \\ 5 \\ 5 \end{bmatrix}.$$

Devemos então resolver:

$$L\mathbf{y} = P\mathbf{b}.$$

Temos:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \frac{1}{2} & -\frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 9 \\ 5 \\ 5 \end{bmatrix}.$$

Da primeira equação,

$$y_1 = 9.$$

Da segunda,

$$y_2 = 5.$$

Da terceira,

$$\frac{1}{2}(9) - \frac{1}{2}(5) + y_3 = 5,$$

portanto,

$$2 + y_3 = 5,$$

e

$$y_3 = 3.$$

Logo,

$$\mathbf{y} = \begin{bmatrix} 9 \\ 5 \\ 3 \end{bmatrix}.$$

Agora resolvemos:

$$U\mathbf{x} = \mathbf{y}.$$

Temos:

$$\begin{bmatrix} 2 & 3 & 4 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 9 \\ 5 \\ 3 \end{bmatrix}.$$

Da terceira equação,

$$z = 3.$$

Da segunda,

$$y + 2(3) = 5,$$

logo,

$$y = -1.$$

Finalmente,

$$2x + 3(-1) + 4(3) = 9,$$

de onde:

$$2x + 9 = 9,$$

e, portanto,

$$x = 0.$$

Assim,

$$x = 0, \quad y = -1, \quad z = 3.$$

Verificação no sistema original:

$$-1 + 2(3) = 5,$$

$$0 + (-1) + 2(3) = 5,$$

e

$$2(0) + 3(-1) + 4(3) = 9.$$

Todas as equações são satisfeitas.



2.9 Considere a matriz:

$$A = \begin{bmatrix} 4 & 1 & 2 \\ 2 & 3 & 1 \\ 1 & 2 & 3 \end{bmatrix}.$$

Realize a decomposição LU da matriz A . Em seguida, utilize a mesma fatoração para resolver, sem refazer a decomposição, os dois sistemas:

$$A\mathbf{x}_1 = \mathbf{b}_1, \quad A\mathbf{x}_2 = \mathbf{b}_2,$$

onde:

$$\mathbf{b}_1 = \begin{bmatrix} 7 \\ 8 \\ 8 \end{bmatrix}, \quad \mathbf{b}_2 = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Ao final, explique por que a fatoração LU é particularmente vantajosa quando vários sistemas possuem a mesma matriz de coeficientes.

Gabarito:

Temos:

$$A = \begin{bmatrix} 4 & 1 & 2 \\ 2 & 3 & 1 \\ 1 & 2 & 3 \end{bmatrix}.$$

Como o primeiro pivô é:

$$a_{11} = 4,$$

e

$$|4| > |2| > |1|,$$

não é necessário pivotamento.

Os multiplicadores da primeira etapa são:

$$\ell_{21} = \frac{2}{4} = \frac{1}{2}, \quad \ell_{31} = \frac{1}{4}.$$

Aplicamos:

$$L_2 \leftarrow L_2 - \frac{1}{2}L_1$$

e

$$L_3 \leftarrow L_3 - \frac{1}{4}L_1.$$

Obtemos:

$$\begin{bmatrix} 4 & 1 & 2 \\ 0 & \frac{5}{2} & 0 \\ 0 & \frac{7}{4} & \frac{5}{2} \end{bmatrix}.$$

Na segunda etapa,

$$\ell_{32} = \frac{\frac{7}{4}}{\frac{5}{2}} = \frac{7}{10}.$$

Assim,

$$L_3 \leftarrow L_3 - \frac{7}{10}L_2.$$

Como o elemento da terceira coluna da segunda linha é zero,

$$a_{33} = \frac{5}{2}.$$

Portanto,

$$U = \begin{bmatrix} 4 & 1 & 2 \\ 0 & \frac{5}{2} & 0 \\ 0 & 0 & \frac{5}{2} \end{bmatrix}$$

e

$$L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{1}{4} & \frac{7}{10} & 1 \end{bmatrix}.$$

Não houve pivotamento, portanto:

$$P = I.$$

Primeiro sistema

Devemos resolver:

$$Ly_1 = b_1.$$

Assim,

$$\begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{1}{4} & \frac{7}{10} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 7 \\ 8 \\ 8 \end{bmatrix}.$$

Da primeira equação,

$$y_1 = 7.$$

Da segunda,

$$\frac{7}{2} + y_2 = 8,$$

logo,

$$y_2 = \frac{9}{2}.$$

Na terceira,

$$\frac{7}{4} + \frac{7}{10} \frac{9}{2} + y_3 = 8.$$

Como:

$$\frac{7}{10} \frac{9}{2} = \frac{63}{20},$$

temos:

$$y_3 = 8 - \frac{7}{4} - \frac{63}{20} = \frac{31}{20}.$$

Portanto,

$$y_1 = \begin{bmatrix} 7 \\ \frac{9}{2} \\ \frac{31}{20} \end{bmatrix}.$$

Agora resolvemos:

$$Ux_1 = y_1.$$

Da terceira equação,

$$\frac{5}{2}z = \frac{31}{20},$$

portanto,

$$z = \frac{31}{50}.$$

Da segunda,

$$\frac{5}{2}y = \frac{9}{2},$$

logo,

$$y = \frac{9}{5}.$$

Finalmente,

$$4x + \frac{9}{5} + 2\frac{31}{50} = 7.$$

Como:

$$\frac{9}{5} + \frac{31}{25} = \frac{45}{25} + \frac{31}{25} = \frac{76}{25},$$

segue que:

$$4x = 7 - \frac{76}{25} = \frac{99}{25},$$

e

$$x = \frac{99}{100}.$$

Logo,

$$\mathbf{x}_1 = \begin{bmatrix} \frac{99}{100} \\ \frac{9}{5} \\ \frac{31}{50} \end{bmatrix}.$$

Segundo sistema

Agora resolvemos:

$$Ly_2 = \mathbf{b}_2.$$

Temos:

$$\begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{1}{4} & \frac{7}{10} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Da primeira equação,

$$y_1 = 1.$$

Da segunda,

$$\frac{1}{2} + y_2 = 2,$$

portanto,

$$y_2 = \frac{3}{2}.$$

Da terceira,

$$\frac{1}{4} + \frac{7}{10} \frac{3}{2} + y_3 = 3.$$

Como:

$$\frac{7}{10} \frac{3}{2} = \frac{21}{20},$$

temos:

$$y_3 = 3 - \frac{1}{4} - \frac{21}{20} = \frac{8}{5}.$$

Logo,

$$\mathbf{y}_2 = \begin{bmatrix} 1 \\ \frac{3}{2} \\ \frac{8}{5} \end{bmatrix}.$$

Agora,

$$U\mathbf{x}_2 = \mathbf{y}_2.$$

Da terceira equação,

$$\frac{5}{2}z = \frac{8}{5},$$

logo,

$$z = \frac{16}{25}.$$

Da segunda,

$$\frac{5}{2}y = \frac{3}{2},$$

portanto,

$$y = \frac{3}{5}.$$

Finalmente,

$$4x + \frac{3}{5} + 2\frac{16}{25} = 1.$$

Assim,

$$4x + \frac{15}{25} + \frac{32}{25} = 1,$$

ou seja,

$$4x + \frac{47}{25} = 1.$$

Logo,

$$4x = -\frac{22}{25},$$

e

$$x = -\frac{11}{50}.$$

Portanto,

$$\mathbf{x}_2 = \begin{bmatrix} -\frac{11}{50} \\ \frac{3}{5} \\ \frac{16}{25} \end{bmatrix}.$$

O ponto fundamental deste exercício é que a fatoração:

$$A = LU$$

foi realizada apenas uma vez. Para cada novo vetor $\mathbf{b}$, são necessárias somente duas substituições triangulares:

$$L\mathbf{y} = \mathbf{b}$$

e

$$U\mathbf{x} = \mathbf{y}.$$

Essa reutilização da fatoração é uma das principais vantagens computacionais da decomposição LU.



2.10 Considere o sistema:

$$\begin{cases} 2x + y - z = 3 \\ 4x + 5y + 2z = 13 \\ -2x + 3y + 4z = 4 \end{cases}$$

Utilizando a decomposição LU:

- (a) determine L e U ;
- (b) resolva o sistema;
- (c) calcule $\det(A)$ a partir da fatoração;
- (d) verifique a solução diretamente no sistema original.

Gabarito:

A matriz dos coeficientes e o vetor de termos independentes são:

$$A = \begin{bmatrix} 2 & 1 & -1 \\ 4 & 5 & 2 \\ -2 & 3 & 4 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 3 \\ 13 \\ 4 \end{bmatrix}.$$

Não é necessário pivotamento, pois o primeiro pivô é:

$$a_{11} = 2 \neq 0.$$

Os multiplicadores da primeira etapa são:

$$\ell_{21} = \frac{4}{2} = 2, \quad \ell_{31} = \frac{-2}{2} = -1.$$

Aplicamos:

$$L_2 \leftarrow L_2 - 2L_1$$

e

$$L_3 \leftarrow L_3 + L_1.$$

Assim,

$$\begin{bmatrix} 2 & 1 & -1 \\ 4 & 5 & 2 \\ -2 & 3 & 4 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & 1 & -1 \\ 0 & 3 & 4 \\ 0 & 4 & 3 \end{bmatrix}.$$

O segundo pivô é:

$$u_{22} = 3.$$

O segundo multiplicador é:

$$\ell_{32} = \frac{4}{3}.$$

Aplicando:

$$L_3 \leftarrow L_3 - \frac{4}{3}L_2,$$

obtemos:

$$\begin{bmatrix} 2 & 1 & -1 \\ 0 & 3 & 4 \\ 0 & 0 & -\frac{7}{3} \end{bmatrix}.$$

Portanto,

$$U = \begin{bmatrix} 2 & 1 & -1 \\ 0 & 3 & 4 \\ 0 & 0 & -\frac{7}{3} \end{bmatrix}$$

e

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & \frac{4}{3} & 1 \end{bmatrix}.$$

Como não houve permutação,

$$P = I.$$

Resolução do sistema triangular inferior

Resolvemos:

$$Ly = b.$$

Temos:

$$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & \frac{4}{3} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 13 \\ 4 \end{bmatrix}.$$

Da primeira equação,

$$y_1 = 3.$$

Da segunda,

$$2(3) + y_2 = 13,$$

portanto,

$$y_2 = 7.$$

Da terceira,

$$-3 + \frac{4}{3}(7) + y_3 = 4.$$

Como:

$$\frac{28}{3} - 3 = \frac{19}{3},$$

temos:

$$\frac{19}{3} + y_3 = 4,$$

e, portanto,

$$y_3 = -\frac{7}{3}.$$

Logo,

$$\mathbf{y} = \begin{bmatrix} 3 \\ 7 \\ -\frac{7}{3} \end{bmatrix}.$$

Agora resolvemos:

$$U\mathbf{x} = \mathbf{y}.$$

Da terceira equação,

$$-\frac{7}{3}z = -\frac{7}{3},$$

logo,

$$z = 1.$$

Da segunda,

$$3y + 4(1) = 7,$$

portanto,

$$y = 1.$$

Finalmente,

$$2x + 1 - 1 = 3,$$

de onde:

$$x = \frac{3}{2}.$$

Assim,

$$\boxed{x = \frac{3}{2}, \quad y = 1, \quad z = 1.}$$

Cálculo do determinante

Como:

$$A = LU,$$

temos:

$$\det(A) = \det(L) \det(U).$$

A matriz L é triangular inferior com diagonal unitária, portanto:

$$\det(L) = 1.$$

Como U é triangular superior,

$$\det(U) = 2 \cdot 3 \cdot \left(-\frac{7}{3}\right).$$

Logo,

$$\det(A) = 2 \cdot 3 \cdot \left(-\frac{7}{3}\right) = -14.$$

Portanto,

$$\boxed{\det(A) = -14}.$$

Como:

$$\det(A) \neq 0,$$

a matriz é não singular e o sistema possui solução única.

Verificação

Substituindo:

$$x = \frac{3}{2}, \quad y = 1, \quad z = 1,$$

na primeira equação:

$$2\left(\frac{3}{2}\right) + 1 - 1 = 3.$$

Na segunda:

$$4\left(\frac{3}{2}\right) + 5 + 2 = 13.$$

Na terceira:

$$-2\left(\frac{3}{2}\right) + 3 + 4 = 4.$$

As três equações são satisfeitas.



2.11 Considere o sistema:

$$\begin{cases} x + 2y + 3z = 6 \\ 2x + 4y + 6z = 12 \\ x + y + z = 3 \end{cases}$$

Utilize a decomposição LU com pivotamento parcial para:

- (a) determinar P , L e U ;
- (b) identificar se a matriz de coeficientes é singular;
- (c) verificar se o sistema possui solução;
- (d) caso possua, determinar o conjunto de soluções;
- (e) explicar por que a decomposição não pode ser utilizada para obter uma solução única.

Gabarito:

A matriz dos coeficientes é:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{bmatrix},$$

e

$$\mathbf{b} = \begin{bmatrix} 6 \\ 12 \\ 3 \end{bmatrix}.$$

1. Primeira etapa do pivotamento

Na primeira coluna temos:

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}.$$

O maior valor absoluto é 2, localizado na segunda linha. Portanto,

$$L_1 \leftrightarrow L_2.$$

A matriz passa a ser:

$$\begin{bmatrix} 2 & 4 & 6 \\ 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix}.$$

Em indexação *zero-based*,

$$\text{ipvt}[0] = 1.$$

O primeiro pivô é:

$$u_{11} = 2.$$

Os multiplicadores são:

$$\ell_{21} = \frac{1}{2}, \quad \ell_{31} = \frac{1}{2}.$$

Aplicando a eliminação,

$$L_2 \leftarrow L_2 - \frac{1}{2}L_1,$$

obtemos:

$$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}.$$

Para a terceira linha,

$$L_3 \leftarrow L_3 - \frac{1}{2}L_1,$$

resultando em:

$$\begin{bmatrix} 0 & -1 & -2 \end{bmatrix}.$$

Assim, a matriz intermediária é:

$$\begin{bmatrix} 2 & 4 & 6 \\ 0 & 0 & 0 \\ 0 & -1 & -2 \end{bmatrix}.$$

2. Segunda etapa do pivotamento

Na segunda coluna, consideramos:

$$\begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

O maior valor absoluto é 1, localizado na terceira linha. Portanto, devemos trocar as linhas 2 e 3:

$$L_2 \leftrightarrow L_3.$$

Obtemos:

$$\begin{bmatrix} 2 & 4 & 6 \\ 0 & -1 & -2 \\ 0 & 0 & 0 \end{bmatrix}.$$

Em indexação *zero-based*,

$$\text{ipvt}[1] = 2.$$

Como a terceira linha já é nula, não existe um terceiro pivô não nulo. Assim,

$$U = \begin{bmatrix} 2 & 4 & 6 \\ 0 & -1 & -2 \\ 0 & 0 & 0 \end{bmatrix}.$$

3. Matriz de permutação

As trocas realizadas foram:

$$L_1 \leftrightarrow L_2$$

e

$$L_2 \leftrightarrow L_3.$$

A ordem final das linhas originais é:

$$(L_2, L_3, L_1).$$

Logo,

$$P = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}.$$

O vetor de pivotamento é:

$$\text{ipvt} = [1, 2, 2].$$

4. Matriz L

Os multiplicadores associados às linhas após o pivotamento são:

$$\ell_{21} = \frac{1}{2}, \quad \ell_{31} = \frac{1}{2}, \quad \ell_{32} = 0.$$

Portanto,

$$L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{1}{2} & 0 & 1 \end{bmatrix}.$$

Podemos verificar a fatora  o:

$$LU = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{1}{2} & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 4 & 6 \\ 0 & -1 & -2 \\ 0 & 0 & 0 \end{bmatrix} \\ = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}.$$

Por outro lado,

$$PA = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{bmatrix} \\ = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}.$$

Portanto,

$$PA = LU.$$

5. Singularidade

A matriz U possui uma linha nula:

$$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}.$$

Consequentemente,

$$\det(U) = 0.$$

Como P e L s o n o singulares,

$$\det(P) \neq 0, \quad \det(L) = 1.$$

Da rela  o:

$$PA = LU,$$

segue que:

$$\det(P) \det(A) = \det(L) \det(U) = 0.$$

Logo,

$$\det(A) = 0.$$

Portanto, A é singular.

6. Análise da consistência do sistema

Devemos agora verificar se o sistema possui solução. Calculamos:

$$P\mathbf{b} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 6 \\ 12 \\ 3 \end{bmatrix} = \begin{bmatrix} 12 \\ 3 \\ 6 \end{bmatrix}.$$

Resolvemos:

$$L\mathbf{y} = P\mathbf{b}.$$

Temos:

$$\begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{1}{2} & 0 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 12 \\ 3 \\ 6 \end{bmatrix}.$$

Da primeira equação,

$$y_1 = 12.$$

Da segunda,

$$6 + y_2 = 3,$$

portanto,

$$y_2 = -3.$$

Da terceira,

$$6 + y_3 = 6,$$

logo,

$$y_3 = 0.$$

Assim,

$$\mathbf{y} = \begin{bmatrix} 12 \\ -3 \\ 0 \end{bmatrix}.$$

Agora devemos resolver:

$$U\mathbf{x} = \mathbf{y},$$

isto é,

$$\begin{bmatrix} 2 & 4 & 6 \\ 0 & -1 & -2 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 12 \\ -3 \\ 0 \end{bmatrix}.$$

As equações são:

$$2x + 4y + 6z = 12,$$

$$-y - 2z = -3,$$

e

$$0 = 0.$$

A última equação não introduz uma restrição adicional. Portanto, existe uma variável livre.

Escolhemos

$$z = t, \quad t \in \mathbb{R}.$$

Da segunda equação,

$$-y - 2t = -3,$$

logo,

$$y = 3 - 2t.$$

Substituindo na primeira equação,

$$2x + 4(3 - 2t) + 6t = 12.$$

Assim,

$$2x + 12 - 8t + 6t = 12,$$

portanto,

$$2x - 2t = 0,$$

e

$$x = t.$$

Consequentemente, o conjunto de soluções é:

$$x = t, \quad y = 3 - 2t, \quad z = t, \quad t \in \mathbb{R}.$$

Ou, na forma vetorial,

$$\mathbf{x} = \begin{bmatrix} 0 \\ 3 \\ 0 \end{bmatrix} + t \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}, \quad t \in \mathbb{R}.$$

Portanto, o sistema possui **infinitas soluções**.

O ponto fundamental deste exercício é que a presença de um pivô nulo na fatoração LU não significa necessariamente que o sistema seja inconsistente. É necessário analisar também o vetor de termos independentes. Neste caso, a linha nula da matriz U corresponde à identidade:

$$0 = 0,$$

e não a uma contradição do tipo:

$$0 = c, \quad c \neq 0.$$

Assim, a matriz é singular, mas o sistema é consistente e possui infinitas soluções.



Implementação em Python

A decomposição LU pode ser implementada diretamente a partir do algoritmo de eliminação de Gauss com pivotamento parcial apresentado anteriormente. Nesta implementação, o procedimento é inteiramente sequencial: em cada etapa, o algoritmo seleciona o pivô, realiza a eventual troca de linhas, calcula os multiplicadores e atualiza, uma a uma, as colunas restantes da matriz.

A implementação utiliza apenas a biblioteca `numpy`, principalmente para a representação das matrizes e para operações elementares sobre seus elementos.

Código: `lu_implementation.py`

```
1 import numpy as np
2
3
4 def build_permutation_matrix(ipvt):
5     n = len(ipvt)
6     P = np.eye(n)
7
8     for k in range(n - 1):
9         i = k
10        j = ipvt[k]
11
12        if i != j:
13            P[[i, j]] = P[[j, i]]
14
15    return P
16
17
18 def lu_factor(A):
19     A = A.copy()
20     n = A.shape[0]
21
22     ipvt = np.arange(n)
23     info = 0
24
25     for k in range(n - 1):
```

```
26
27     # Pivotamento parcial
28     pivot_row = k + np.argmax(np.abs(A[k:, k]))
29     ipvt[k] = pivot_row
30
31     # Troca de linhas
32     if pivot_row != k:
33         A[[k, pivot_row], :] = A[[pivot_row, k], :]
34
35     # Verificacao do pivo
36     if A[k, k] == 0:
37         info = k
38         continue
39
40     # Calcula os multiplicadores
41     A[k+1:, k] /= A[k, k]
42
43     # Atualiza as colunas seguintes
44     for j in range(k + 1, n):
45         A[k+1:, j] -= A[k, j] * A[k+1:, k]
46
47     # Verificacao do ultimo pivo
48     if A[n-1, n-1] == 0:
49         info = n - 1
50
51     # Separa L e U
52     L = np.tril(A, k=-1) + np.eye(n, dtype=A.dtype)
53     U = np.triu(A)
54
55     return L, U, ipvt, info
56
57
58 # Matriz 3x3 inteira para facilitar o entendimento
59 A = np.array([
60     [2, 3, 1],
61     [4, 7, 7],
62     [6, 18, 22]
63 ], dtype=float)
64
```

```
65
66 L, U, ipvt, info = lu_factor(A)
67
68
69 print("Matriz A original:")
70 print(A)
71
72 print("\nL (triangular inferior):")
73 print(L)
74
75 print("\nU (triangular superior):")
76 print(U)
77
78 print("\nPivot indices:")
79 print(ipvt)
80
81 print("\nInfo:")
82 print(info)
83
84
85 # Verificacao: P @ A = L @ U
86 P = build_permutation_matrix(ipvt)
87
88 reconstructed = P @ A
89 LU = L @ U
90
91 print("\nP @ A:")
92 print(reconstructed)
93
94 print("\nL @ U:")
95 print(LU)
96
97 print("\nErro (norma de Frobenius):",
98       np.linalg.norm(reconstructed - LU))
```

Referências Bibliográficas

- [1] BURDEN, R. L.; FAIRES, J. D. *Análise Numérica*. 9. ed. São Paulo: Cengage Learning, 2011.
- [2] VIRTANEN, P.; OUTROS. *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. 2020. <https://scipy.org>. Acesso em: 13 ago. 2025.
- [3] MEYER, C. D. *Matrix Analysis and Applied Linear Algebra*. Philadelphia: SIAM, 2000.
- [4] STRANG, G. *Introduction to Linear Algebra*. [S.l.]: Wellesley-Cambridge Press, 2009.
- [5] GOLUB, G. H.; LOAN, C. F. V. *Matrix Computations*. [S.l.]: Johns Hopkins University Press, 1996.
- [6] BURDEN, R. L.; FAIRES, J. D. *Numerical Analysis*. 11. ed. [S.l.]: Cengage Learning, 2022.
- [7] CHAPRA, S. C.; CANALE, R. P. *Numerical Methods for Engineers*. 6. ed. [S.l.]: McGraw-Hill Education, 2010.
- [8] ANTON, H.; RORRES, C. *Elementary Linear Algebra: Applications Version*. 10. ed. Hoboken: John Wiley & Sons, 2010. ISBN 9780470432051.
- [9] GOLUB, G. H.; LOAN, C. F. V. *Matrix Computations*. 4. ed. Baltimore: The Johns Hopkins University Press, 2013. ISBN 9781421408590.
- [10] TREFETHEN, L. N.; III, D. B. *Numerical Linear Algebra*. 25th anniversary. ed. Philadelphia: Society for Industrial and Applied Mathematics, 2022. ISBN 9781611977158.